

重庆邮电大学

CHONGQING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

硕士学位论文

MASTER THESIS



论文题目 极化码低复杂度 SCL 译码算法
 的优化研究

学科专业 电子科学与技术

学 号 S230401049

作者姓名 杨德意

指导教师 袁建国 教授

学 院 电子科学与工程学院

学校代码	10617	UDC	621.39
分类号	TN911.22	密级	公开

学 位 论 文

极化码低复杂度 SCL 译码算法的优化研究

杨德意

指导教师 袁建国 教授

申请学位级别	硕士	学科专业	电子科学与技术
答辩委员会主席		论文答辩日期	2026 年 5 月 23 日
学位授予单位和日期	重庆邮电大学		

Optimization Research on the Low Complexity Decoding Algorithm of Polar Codes

A Master Thesis Submitted to
Chongqing University of Posts and Telecommunications

Discipline **Electronic Science and Technology**

Student ID **S230401049**

Author **YANG Deyi**

Supervisor **Prof. YUAN Jianguo**

School **School of Electronic Science and Engineering**

重庆邮电大学

学位论文独创性声明

本人郑重声明：所呈交的学位论文，是本人在导师指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文中不包含其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在论文中以明确方式标明。本人完全知晓本声明的法律后果由本人承担。

学位论文作者签名：

日期： 年 月 日

重庆邮电大学

学位论文使用授权书

本人同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。

本学位论文属于：

☐ 公开论文

☐ 涉密论文，保密____年，过保密期后适用本授权书。

（请在以上方框内选择打“√”）

作者签名：

导师签名：

日期： 年 月 日

摘 要

极化码在循环冗余校验辅助的串行抵消列表 (Cyclic Redundancy Check Aided Successive Cancellation List, CA-SCL) 译码算法提出后, 其性能得到了显著提升, 因此 CA-SCL 成为了重要的基线算法。但列表的增加也为其带来了明显的复杂度提升, 因此降低串行抵消列表 (Successive Cancellation List, SCL) 译码算法的复杂度成为了重要的研究方向。本文针对极化码低复杂度 SCL 译码算法进行研究和改进。本文研究内容如下:

1、针对基于搜索集的路径分裂策略辅助 SCL (Path Splitting Selecting Strategy based on Search Set under the Successive Cancellation List, PSS-SS-SCL) 译码算法存在的冗余路径分裂问题, 本文提出一种基于可靠性函数的路径选择策略 SCL (Path Splitting Selecting strategy based on Reliability Function under the Successive Cancellation List, PSS-RF-SCL) 译码算法。该算法在信息比特进行路径分裂前, 先基于当前路径度量 (Path Metric, PM) 值计算比特的可靠性函数值, 并与设定的阈值 α 比较, 若低于 α 则判定为不可靠比特, 执行路径分裂。反之则进行硬判决, 提升译码效率。同时, 为有效控制路径数量, 算法引入辅助路径度量 (Auxiliary Path Metric, APM) 值作为路径是否错误的评估指标, 并将其与阈值 β 比较。当某条路径的 APM 值高于 β 时, 判定为不可靠路径并进行剪枝, 从而在保留正确路径的同时降低列表规模。仿真结果表明, 与现有的各种基于搜索集的路径分裂策略辅助 SCL 译码算法相比, 本文算法在没有性能损失的前提下, 有效降低了译码复杂度。

2、针对快速简化 SCL (Fast Simplify Successive Cancellation List, Fast-SCL) 译码算法在 Rate-1 节点中的局限性和在并行路径分裂中存在的问题, 本文提出一种候选路径选择策略辅助并行分裂简化译码算法 (Path Selection strategy assisted Parallel Split and Simplify Successive Cancellation List, PS-PSSCL)。该算法在保持并行度的前提下, 通过优化 Rate-1 节点的路径分裂机制来降低候选路径数量。具体而言, 采用并行路径分裂策略将多个比特的路径拓展压缩至单一时间步长内完成, 以降低译码延迟。同时采用融合路径偏序关系与翻转比特对数似然比之和的筛选机制, 在不影响纠错性能的前提下降低候选路径的数量。仿真结果表明, 与同类并行分裂算法相比, 所提算法在保持几乎无损译码性能的同时, 能以更低的候选路径数量完成并行分裂, 缓解排序、删除及存储压力, 有效降低译码复杂度。

关键词: 极化码, 串行抵消列表, 低复杂度, 快速串行抵消列表, 路径分裂

ABSTRACT

After the introduction of the Cyclic Redundancy Check Aided Successive Cancellation List (CA-SCL) decoding algorithm, the performance of polar codes has been significantly improved, making CA-SCL an important baseline algorithm. However, an increase in the list size leads to a substantial rise in complexity. Therefore, reducing the complexity of the SCL decoding algorithm has become an important research direction. This thesis focuses on the research and improvement of the low-complexity SCL decoding algorithm for polar codes. The research contributions are as follows:

1. To address the issue of redundant path splitting in the Path Splitting Selecting Strategy based on Search Set under the Successive Cancellation List (PSS-SS-SCL) decoding algorithm, this thesis proposes a Path Splitting Selecting strategy based on Reliability Function under the Successive Cancellation List (PSS-RF-SCL) decoding algorithm. Before splitting paths for an information bit, this algorithm first calculates the bit's reliability function value based on the current Path Metric (PM). This value is compared against a set threshold α , if it falls below α , the bit is determined as unreliable and path splitting is executed. Otherwise, a hard decision is made to improve decoding efficiency. Meanwhile, to effectively control the number of paths, the algorithm introduces the Auxiliary Path Metric (APM) as an indicator for determining whether a path is incorrect, and compares it with the threshold β . When a path's APM value exceeds β , it is determined as an unreliable path and pruned, thus reducing the list size while retaining the correct path. Simulation results show that, compared with existing path splitting strategies based on search sets aiding SCL decoding algorithms, the algorithm in this thesis effectively reduces decoding complexity without performance loss.

2. Addressing the limitations of the Fast Simplify Successive Cancellation List (Fast-SCL) decoding algorithm in Rate-1 nodes and the problems encountered in parallel path splitting, this thesis proposes a Path Selection strategy assisted Parallel Split and Simplify Successive Cancellation List (PS-PSSCL) decoding algorithm. This algorithm reduces the number of candidate paths by optimizing the path splitting mechanism of Rate-1 nodes, while maintaining parallelism. Specifically, the parallel path splitting strategy is adopted to compress the path extension of multiple bits into a single time step to reduce decoding delay. At the same time, a filtering mechanism that combines path partial order relations

with the sum of log-likelihood ratios of flipped bits is employed to reduce the number of candidate paths without compromising error correction performance. Simulation results demonstrate that, compared to similar parallel splitting algorithms, the proposed algorithm achieves parallel splitting with fewer candidate paths while maintaining virtually lossless error performance. This alleviates the pressure of sorting, pruning and storage, reducing decoding complexity.

Keywords: Polar Codes, Successive Cancellation List, Low Complexity, Fast Successive Cancellation List, Path Splitting

目 录

摘 要	I
ABSTRACT	II
图目录	VI
表目录	VIII
第 1 章 绪论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	2
1.3 论文研究主要内容与组织结构	7
第 2 章 极化码基础理论分析	9
2.1 本章引言	9
2.2 信道极化理论	9
2.2.1 信道的参数	9
2.2.2 信道合并	10
2.2.3 信道分裂	13
2.3 极化码编码	14
2.3.1 极化子信道的可靠性估计	14
2.3.2 极化码编码过程	17
2.4 极化码译码	18
2.4.1 SC 译码算法	18
2.4.2 CA-SCL 译码算法	20
2.5 本章小结	23
第 3 章 基于可靠性函数的极化码 PSS-RF-SCL 译码算法研究	24
3.1 本章引言	24
3.2 PSS-SS-SCL 译码算法	24
3.3 降低复杂度的译码策略	26
3.3.1 基于可靠性函数的路径分裂策略	27
3.3.2 路径剪枝策略	28
3.4 一种基于可靠性函数的极化码 PSS-RF-SCL 译码算法	30
3.4.1 PSS-RF-SCL 译码算法的译码过程	31
3.4.2 仿真分析	32

3.5 本章小结	36
第 4 章 候选路径选择策略辅助并行分裂简化译码算法的研究	38
4.1 本章引言	38
4.2 Fast-SCL 译码算法	38
4.3 一种候选路径选择策略辅助并行分裂简化译码算法	40
4.3.1 并行分裂方式	40
4.3.2 筛选候选路径的方式	41
4.3.3 PS-PSSCL 译码算法流程	45
4.3.4 仿真分析	48
4.4 本章小结	55
第 5 章 总结与展望	56
5.1 主要工作及结论	56
5.2 研究展望	57
参考文献	59
作者简介	64
1. 基本情况	64
2. 教育和工作经历	64
3. 攻读学位期间的研究成果	64
3.1 发表的学术论文和著作	64
3.2 申请（授权）专利	64
3.3 参与的科研项目及获奖	64
致 谢	65

图目录

图 1-1 本文各章节研究内容及逻辑关系	7
图 2-1 码长 512 的极化码信道容量分布	10
图 2-2 信道 w_2 示意图	11
图 2-3 信道 w_4 示意图	11
图 2-4 N 个信道合并示意图	12
图 2-5 信道 w_8 的分裂示意图	13
图 2-6 SC 译码前	19
图 2-7 SC 译码后	19
图 2-8 SCL 译码树	20
图 2-9 CA-SCL 译码算法流程图	21
图 2-10 经典译码算法的误块率仿真对比图	22
图 3-1 PSS-SS-SCL 译码算法的译码流程图	25
图 3-2 极化码译码二叉树示意图	25
图 3-3 不同码长和信噪比下被淘汰的正确路径的 APM 值分布情况	29
图 3-4 PSS-RF-SCL 译码算法的整体流程图	31
图 3-5 $N=1024, R=1/2, L=8$ 时各算法误块率仿真图	33
图 3-6 $N=1024, R=1/2, L=16$ 时各算法误块率仿真图	34
图 3-7 $N=512, R=1/2, L=8$ 时各算法误块率仿真图	34
图 3-8 不同算法的平均译码列表总数	35
图 3-9 不同算法的平均分裂次数	36
图 4-1 四种特殊节点的译码二叉树	39
图 4-2 路径分裂方式	41
图 4-3 最好排名表	42
图 4-4 相同翻转数量下的偏序关系	43
图 4-5 最好排名表	43
图 4-6 最好排名表	45
图 4-7 并行分裂译码结构	47
图 4-8 $N=1024, R=1/2, L=8$ 时各算法误块率仿真图	48
图 4-9 $N=1024, R=1/2, L=4$ 时各算法误块率仿真图	49
图 4-10 $N=1024, R=3/4, L=8$ 时各算法误块率仿真图	50

图 4-11 $N=512, R=1/2, L=8$ 时各算法误块率仿真图	50
图 4-12 $N=256, R=1/2, L=8$ 时各算法误块率仿真图	51
图 4-13 $N=1024, R=1/2$ 时不同译码算法在瑞利衰落信道下的误块率仿真图	54

表目录

表 2-1 码长为 8 的极化权重 $PW(i)$	17
表 3-1 阈值 α 的值	28
表 3-2 阈值 β 的值.....	30
表 4-1 码长 $N=1024$ 时各算法的时间步长	52
表 4-2 各算法在 Rate-1 节点中的候选路径数量.....	53

第 1 章 绪论

1.1 研究背景及意义

随着移动通信技术的飞速发展，人类社会正以前所未有的速度迈向万物互联的智能化时代。从移动支付、线上会议到远程医疗和自动驾驶，通信技术已深度渗透至人们生活的方方面面，成为现代社会运转的重要基石。通信系统的性能直接关系到信息传递的安全性、可靠性与高效性，而信道编码技术作为保障信息可靠传输的关键手段，始终是通信领域研究的核心内容之一。

1948 年，Shannon 提出了著名的有噪信道编码定理，指出只要信息传输速率不超过信道容量，总存在一种编码方式使得传输误差概率任意小。这一理论为信道编码研究奠定了坚实的数学基础，也开启了全球学者对“好码”的持续探索之路。早期的信道编码以代数编码为主，如汉明码、格雷码、循环码等，这些编码在给定码长和信息比特数下通过最大化码字间的最小汉明距离来提高纠错能力。然而，受限于编码结构和译码复杂度，这些编码方案与 Shannon 极限之间存在明显差距。

20 世纪 50 年代，卷积码的出现为信道编码带来了新的思路。与分组码相比，卷积码能够利用前后数据块之间的相关性，降低了系统延时和同步要求。随后，Turbo 码和低密度奇偶校验（Low-density Parity-check, LDPC）码的相继提出，标志着信道编码研究进入了一个全新阶段。Turbo 码通过卷积码的级联与迭代译码，在加性高斯白噪声（Additive White Gaussian Noise, AWGN）信道下实现了逼近香农极限的优异性能，成为 3G 和 4G 移动通信系统的核心编码方案。LDPC 码则凭借其低译码复杂度和接近香农极限的性能，在数字电视、WiFi、卫星通信等领域得到了广泛应用。尽管 Turbo 码和 LDPC 码的性能已极为接近香农极限，但二者均无法从理论上被证明能够真正达到这一极限。

2009 年，土耳其教授 Arıkan 在国际信息论研讨会上首次提出了基于信道极化现象的极化码编码方案，并在后续论文中系统阐述了其编译码原理^[1]。极化码的核心思想在于通过信道合并与分裂，将若干独立信道极化为容量接近 1 的可靠信道和容量接近 0 的不可靠信道，从而实现信息比特在可靠信道上的高效传输。Arıkan 从理论上首次严格证明，在码长趋于无穷大时，极化码能够在二进制离散无记忆信道下达到香农极限。这一突破性成果使得极化码成为信道编码领域历史上首个被严格证明可达香农极限的编码方案。

极化码不仅具有理论上的优越性，其实用价值同样显著。2016 年，极化码在 3GPP 会议上被正式确定为 5G 增强移动带宽场景中控制信道的编码方案，标志着

极化码从理论研究走向实际应用,展现出巨大的发展潜力。与 Turbo 码和 LDPC 码相比,极化码在串行抵消 (Successive Cancellation, SC) 译码算法下具有较低的编译码复杂度,且不存在错误平层现象,码长和码率的灵活性也更强。但 SC 译码算法的性能并不够理想,在中短码长下与 LDPC 码相比有明显差距。但在循环冗余校验辅助的串行抵消列表 (Cyclic Redundancy Check Aided Successive Cancellation List, CA-SCL) 译码算法被提出后^[2],其性能可以与 LDPC 码相竞争。但列表数的增加使得串行抵消列表 (Successive Cancellation List, SCL) 译码算法的时间复杂度和空间复杂度都明显上升,这带来了较高的时延和硬件性能需求。信道编译码算法作为通信系统中的关键技术,其复杂度直接影响系统的传输效率。低复杂度的信道编译码算法能够降低系统的处理时延和功耗,从而提高系统的整体性能。同时,较低的复杂度也意味着算法更易于在硬件上实现,有助于降低系统的硬件成本。

综上所述,极化码作为信道编码领域的重大突破,不仅在理论上实现了香农极限的可达性,也在 5G 及未来通信系统中展现出广阔的应用前景。在 CA-SCL 作为主流译码算法的情况下,降低其复杂度对实现高吞吐率和低延迟的极化码译码有相当重要的作用。因此对极化码的低复杂度 SCL 译码算法的研究是非常有必要的。

1.2 国内外研究现状

极化码的编码过程主要包括生成矩阵的构造和信息位的选取两个环节。其中,生成矩阵通常由二维核矩阵通过克罗内克积运算得到,其构造方式相对固定。因此,极化码编码性能的差异主要体现为信息位选取策略的不同,即如何准确评估各极化子信道的可靠性。针对不同的信道环境,研究者们提出了多种可靠性估计方案。Arikan 教授在提出极化码时,针对二进制擦除信道 (Binary Erasure Channel, BEC) 给出了巴氏参数法^[1]。该方法利用极化码的结构特性,通过递归方式计算各子信道的巴氏参数,以此作为信道可靠性的度量,巴氏参数越大表示信道可靠度越低。巴氏参数法在 BEC 信道下具有精确的解析表达式,能够准确选取可靠子信道。然而,对于其它信道类型,巴氏参数只能近似估计,且计算复杂度较高,因此其适用范围受限。

为了获得更通用的构造方法,蒙特卡洛法被早期研究者采用。该方法通过大量仿真统计各子信道的译码错误概率,从而选择出可靠性较高的子信道。蒙特卡洛法能够适用于任意信道,但其时间复杂度随码长增加呈指数级增长,当码长较长或需要高精度时,仿真次数巨大,难以在实际工程中推广使用。

2009 年, Mori 等人将 LDPC 码的密度进化 (Density Evolution, DE) 理念引入极化码,提出了一种基于密度进化的构造方法^[3]。该方法利用概率密度函数递归计

算每个子信道的错误概率，适用于一般的二进制输入离散无记忆对称信道（Binary Discrete Memoryless Channel, B-DMC）。密度进化法能够精确评估信道可靠性，但涉及大量的卷积运算，计算复杂度很高。尽管后续研究引入了量化和降级量化策略^[4]以降低复杂度，但整体计算量依然较大，限制了该方法的应用。针对密度进化法计算复杂的问题，高斯近似（Gaussian Approximation, GA）法^[5]被提出。该方法假设接收信号的对数似然比（Logarithm Likelihood Ratio, LLR）在极化过程中始终服从高斯分布，从而将复杂的卷积运算简化为均值和方差的递推计算。GA 法在加性高斯白噪声信道下具有较低的复杂度和良好的性能，成为目前学术界广泛采用的构造方法之一。码长为 N 的极化码使用 GA 法选取信息位的复杂度为 $o(N)$ ，且在中短码长下能够获得较好的纠错性能。

为进一步降低构造复杂度并摆脱对信道参数的依赖，华为公司在 3GPP 会议上提出了极化权重（Polarization Weight, PW）法^[6]。极化权重法是一种与信道参数无关的信息位选取策略。该方法首先将信道索引的十进制序号转换为二进制形式，随后以 $2^{0.25}$ 作为基准值来计算各极化信道的权重值，并依据权重高低筛选出信息位。由于该方法不依赖具体的信道状态信息，其判定准则可预先固化为一个通用序列，适用于任意码长和码率的情况。实验表明，PW 法在不同参数环境下均能维持稳定且优异的性能。最终，PW 法被 3GPP 确定为 5G 标准协议中极化码的构造方法。

极化码的译码算法是决定其误码性能、计算复杂度和译码时延的关键因素。自极化码被提出以来，学者们围绕译码算法开展了大量研究，形成了以 SC 译码算法为基础，向性能提升、复杂度降低、时延优化等多个方向拓展的研究格局。SC 译码算法是 Arikan 教授提出极化码的同时给出的译码算法。SC 译码器按照比特序号顺序逐位进行译码，每个比特的译码结果依赖于接收信号及前序比特的估计值，其复杂度为 $o(N \cdot \log N)$ 。在码长趋于无穷时，SC 译码算法具有最大似然译码性能。然而，由于实际系统中码长有限，信道极化不充分，SC 译码算法在中短码长下的误块率（Block Error Rate, BLER）表现较差，落后于同期的主流编码，如 LDPC 码和 Turbo 码。在 SC 译码算法的基础上，学者们从减少译码复杂度和提升性能两个方向进行了改进。Sarkis 等人提出了快速 SC（Fast-SC）译码算法，通过识别 Rate-0 节点、Rate-1 节点^[7]、单奇偶校验（SPC）节点和重复（REP）节点^[8]并采用特殊译码规则，避免了遍历译码树中的冗余节点，从而降低了译码复杂度。文献[9]归纳了这些特殊节点结合奇偶校验（Parity Check, PC）码的方式，设计了相应的快速译码方案。针对硬件实现，文献[10]提出了基于均匀量化的快速 SSC 译码算法，在保留整数型运算的同时引入特殊节点，有效节省了存储空间和计算资源。

为了解决 SC 译码算法在有限码长下性能不足的问题, Tal 和 Vardy 提出了 SCL 译码算法^[11]。SCL 译码器在译码过程中保留 L 条译码路径, 在每个信息比特处进行路径扩展, 并依据路径度量 (Path Metric, PM) 值保留最优的 L 条路径, 译码结束后选择 PM 值最小的路径作为输出。SCL 译码算法在 L 较大时性能接近最大似然译码, 但其复杂度也随 L 线性增长, 当码长为 N 时, 其时间复杂度为 $O(L \cdot N \cdot \log N)$ 。该算法的核心思想在于通过保留多条译码路径来弥补 SC 译码中早期错误决策无法纠正的缺陷, 在译码每个信息比特时将当前每条路径分别扩展为对应比特 0 和 1 的两条新路径, 再根据 PM 值筛选出度量最小的 L 条路径进入下一比特的译码, 这种路径竞争机制有效降低了错误传播概率, 使得算法能够在全局范围内寻找最优译码路径, 同时 L 条路径的存储需求对硬件实现提出了更高要求, 因此在应用中需要在译码性能和实现复杂度之间进行权衡。

为进一步提升 SCL 译码算法的误块率性能, Niu 等人利用循环冗余校验 (Cyclic Redundancy Check, CRC) 码进行校验, 提出了循环冗余校验辅助的 SCL (CRC-Aided SCL, CA-SCL) 译码算法。该算法在编码端将 CRC 比特附加于信息比特之后, 在译码端利用 CRC 校验从 L 条候选路径中筛选通过校验且 PM 值最小的路径作为最终输出。CA-SCL 译码算法大幅提升了极化码在中短码长下的纠错性能, 使其能够与 LDPC 码和 Turbo 码相媲美, 并被 5G 标准采纳为控制信道的译码方案。传统 CRC 辅助译码方案中, 校验码位于信息序列末端, 导致译码过程中的错误无法被及时发现。为克服这一局限, 文献[12]引入了分段 CRC 辅助 SCL (Segment CRC Aided SCL, SCA-SCL) 译码算法。该算法将信息序列划分为多个子块, 并为每个子块独立添加 CRC 校验码。译码时, 每完成一个子块的译码便立即执行 CRC 校验, 一方面能够及时剪枝已偏离正确结果的候选路径, 另一方面可在所有路径均未通过校验时提前终止译码进程。这种分段校验机制有效避免了无效的后继计算, 降低了译码复杂度。

近年来, 校验辅助技术已成为提升译码准确性的重要手段之一。除了 CRC 码外, PC 码也因为其在辅助极化码译码中的良好表现而备受关注。为充分发挥两类校验码的互补优势, 学者们提出了一种 CRC 码与 PC 码联合校验的方案^[13]。该方案在 SCL 译码的过程中, 利用分散布置的 PC 码对候选路径进行早期剪枝, 及时剔除不符合校验约束的错误路径, 能够有效控制译码复杂度的增长。在译码结束时, 借助 CRC 码从幸存路径中筛选出最符合全局校验约束的路径作为最终输出。仿真结果表明, 联合校验方案在误码性能上优于单一使用 CRC 码或 PC 码进行校验的方案。为进一步提升校验的针对性与纠错能力, 文献[14]提出了一种分布式 PC 码方案, 将 PC 码非均匀地嵌入信息序列中, 使其更精准地定位和纠正译码过程中可

能出现的错误。研究表明，在信息段中引入多个奇偶校验码对同一段信息进行多重校验，能够明显增强校验的有效性。在该方法的基础上，文献[15]进一步提出了双奇偶校验辅助 CA-SCL (Double PC Aided CA-SCL, DP-CA-SCL) 译码算法，通过在信息序列中设置双重奇偶约束，在译码过程中实现更严格的路径筛选，从而在保持较低译码复杂度的前提下进一步提升译码性能。这些研究表明，合理设计和布局校验结构是推动极化码实用化的重要方向之一。

为了降低 SCL 译码算法的高复杂度，学者们从两个不同角度提出了优化方案：一是利用特殊节点实现快速译码，二是通过路径分裂选择策略减少不必要的路径分裂操作。这两种方法分别从两个方面降低了极化码译码复杂度。特殊节点快速译码的核心思想是在译码过程中识别具有特定冻结比特分布模式的节点，并为其设计高效的并行译码规则，从而避免逐比特遍历带来的高时延。早期这种方案是应用于 Fast-SC 译码算法。然而，Fast-SC 算法与 SC 译码算法类似，在译码过程中始终只有一条译码路径，仍存在错误传播问题导致其性能不理想。为此，学者们进一步将这种特殊节点快速译码的思想引入 SCL 译码算法当中。提出了快速简化 SCL (Fast Simplify SCL, Fast-SCL) 译码算法^[16-18]，该算法在保留多条译码路径的同时引入特殊节点快速译码机制，相较于 CA-SCL 译码算法，在保持性能没有降低的情况下明显降低了译码复杂度。文献[19]提出了五种新节点 Type I~V，随后这五种节点也被运用于简化 SCL 译码算法^[20]。文献[21]提出了序列重复 (Sequence Repetition, SR) 节点，该节点可支持广泛类型的节点处理。并且后续提出了分阶段的译码方案以及基于 SCL 译码算法的自适应路径分裂策略^[22]。文献[23]在 Fast-SCL 译码算法的基础上，针对 Rate-1 节点设计了一种并行译码方式，解决了多比特翻转的串行问题。后续有学者对其进行了优化，并参考该思想进行了 SPC 节点的并行译码简化^[24-25]。文献[26]生成的翻转组合 (Minimum-Combinations, MC) 比特集合也达到了将 Rate-1 节点的比特翻转次数降低为一的目的。简化译码也被用来与比特翻转算法相结合。串行抵消列表比特翻转 (Successive Cancellation List Bit-flipping, SCLF) 译码算法是在译码错误后选取特定位置进行翻转，通过纠正首个译码错误达到译码性能提升的目的^[27-28]。文献[29]将运用特殊节点的简化译码与比特翻转相结合，得到了比 CA-SCL 译码算法更好的误块率性能，同时复杂度损失适中。在翻转比特方向上，还有动态 SCLF 译码算法^[30]。并且文献[31]提出了归一化置信度，通过该置信度构造搜索集进一步提升了对错误比特的搜索能力，实现了误块率性能的提升。文献[32]对 SCLF 译码算法进行了改进，其核心在于密钥集的构造与优化策略。与传统的静态搜索集不同，该密钥集从首次 CA-SCL 译码结果中

动态提取,能够更准确地反映错误比特的分布。此外,算法借助高斯近似动态调节密钥集的大小,使得缩减集合大小的同时精准地定位不可靠比特,进而降低复杂度。

除了针对特殊节点,还有针对译码路径筛选的简化方案。有学者利用极化码不同比特之间可靠性的显著差异,提出对比特进行可靠性估计,只对可靠性较低的比特进行路径分裂,对于可靠性较高的比特直接采用硬判决译码可以减少一部分路径的译码操作,从而有效减少译码复杂度,且性能损失较小。SCL 译码算法的计算复杂度受路径分裂和排序操作的影响不小,尤其是在列表较大的情况下,这些操作的开销十分昂贵。路径分裂选择可以减少路径的分裂和排序次数,降低计算复杂度。该方案的关键在于如何选取需要执行路径分裂的比特位置集合。文献[33]中介绍了一种基于搜索集的路径分裂策略辅助串行抵消列表(Path Splitting Selecting Strategy based on Search Set under the Successive Cancellation List, PSS-SS-SCL)译码算法,该算法基于路径分裂选择策略,并借助搜索集的帮助来实现复杂度的降低。该算法通过提取所有 Rate-1 节点中的首个比特构建搜索集,从而在译码过程中更高效地进行路径扩展。实验结果表明,该算法在保证对误块率影响较小的前提下,能够明显降低算法的整体计算复杂度。此外,不同搜索集的构造方案可根据实际需求在性能与复杂度之间进行权衡。部分路径分裂策略通过精准定位需要分裂的比特,有效减少了路径扩展和排序带来的计算负担,为 SCL 译码算法在中短码长下的实际应用提供了有力支持。文献[34]通过信道的偏序关系提出了数种搜索集的构造方式。文献[35]则提出了一种考虑了相邻比特和错误传播影响的优化关键级(Improve Critical Set, ICS),ICS 相较于搜索级的定位更加精准。上述方案只考虑了比特的可靠性,并没有考虑译码路径的可靠性,因此有学者进一步考虑到不可靠路径可以不进行路径分裂^[36]。

此外,路径数量与采用的列表数有明显关系。如果采用较大列表数,在低信噪比下的性能提升并不会很明显,但复杂度却会明显上升。如果采用较小的列表数,在高信噪比时,性能却会有明显损失。因此一些学者提出了列表数随着信道环境进行调整的自适应 SCL(Adaptive SCL, AD-SCL)译码算法^[37-38]。在文献[39]中,该算法先设置较大的列表数进行译码,当译完某段并且校验正确后,此时将列表数降低进行后续译码,以此降低译码路径数。文献[40]提出了奇偶校验辅助自适应 SCL(Parity Check Aided AD-SCL, PC-ASCL)译码算法,该译码算法用 PC 码来校验当前段译码序列,降低了译码复杂度。

除了前文所述的译码算法外,极化码还存在置信传播(Belief Propagation, BP)译码算法^[41-44]、球译码算法^[45-47]、基于列表的球译码算法^[48]、深度学习辅助的译码算法^[49-52]、以及 SCL 扰动译码算法^[53-55]。

综上所述，极化码在 CA-SCL 译码算法被提出后性能已经可以与 LDPC 码等相互媲美。但随着列表数的增加，其译码复杂度明显上升，带来了较高的时延和硬件性能需求。在 CA-SCL 译码算法被作为主流基础算法的情况下，降低其译码复杂度对实现高吞吐率和低延迟的极化码译码器有相当重要的作用，也是极化码目前的研究热点之一。但目前低复杂度的 SCL 译码算法还存在以下问题：1. PSS-SS-SCL 译码算法的搜索集是按照经验分析得来，造成了多余的分裂次数，并且对于不可靠路径没有提前剪枝，造成了多余的译码路径数。2. 在 Fast-SCL 译码算法的 Rate-1 节点中实行并行分裂后，候选路径数量过多。本文将针对低复杂度 SCL 译码算法的这些问题进行研究改进。

1.3 论文研究主要内容与组织结构

本文主要针对低复杂度 SCL 译码算法的两个方案进行分析和研究。分别是译码路径筛选方案和基于特殊节点的简化译码方案。本文各章节研究内容及逻辑关系如图 1-1 所示。

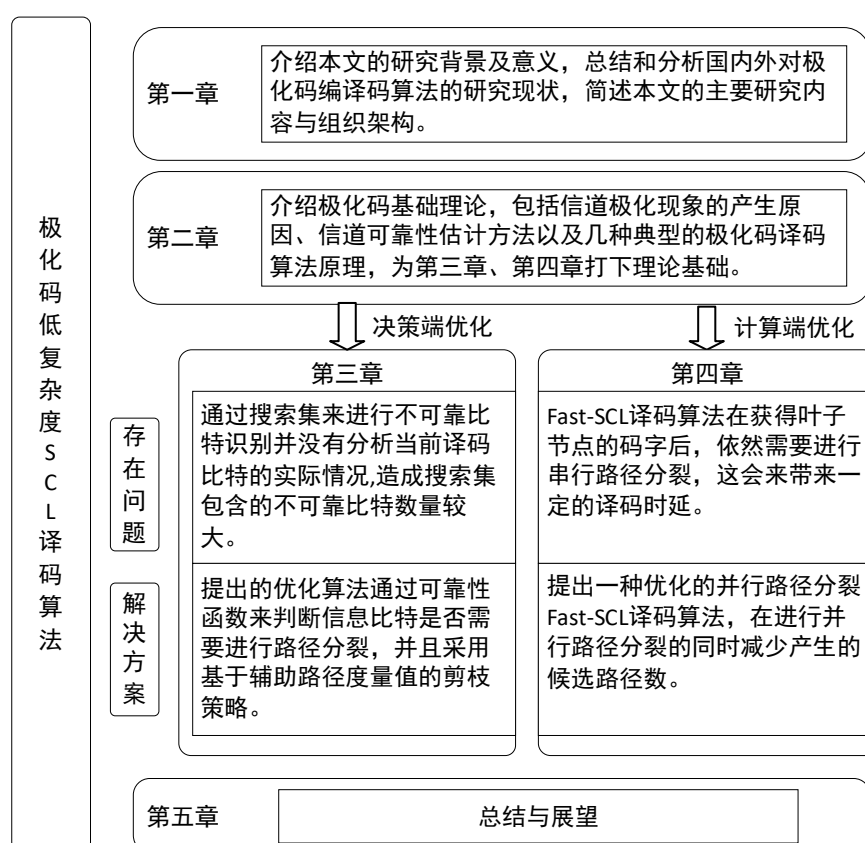


图 1-1 本文各章节研究内容及逻辑关系

Fig.1-1 Research contents and logical relationships of each section in this paper

本文主要内容安排如下：

第一章：绪论。本章首先阐述了信道编码的基本功能及其演进历程，并重点分析了极化码的优越性能及其理论价值。随后，对国内外在极化码编译码算法领域的研究进展进行了综述，最后介绍了全文的组织结构。

第二章：极化码的基本原理。本章首先对极化信道的相关参数进行了介绍，并详细讨论了信道的合并与分裂过程，进而阐释了极化定理的核心思想。在此基础上，介绍了极化码编码流程中至关重要的信道估计方法及其实现原理。最后，系统阐述了经典的串行抵消译码算法以及应用广泛的 CA-SCL 译码算法，为后续研究奠定理论基础。

第三章：基于可靠性函数的路径选择策略 SCL 译码算法研究。该章是对 SCL 译码算法的决策端进行优化。首先介绍了采用路径筛选方案的 PSS-SS-SCL 译码算法，分析了其进步之处以及存在的问题。然后针对这些问题提出了改进的降低复杂度的译码策略，分别是一种新的路径分裂策略和剪枝策略。将这些策略用于优化 PSS-SS-SCL 译码算法，提出了基于可靠性函数的路径选择策略 SCL (Path Splitting Selecting strategy based on Reliability Function under the Successive Cancellation List, PSS-RF-SCL) 译码算法。该算法通过可靠性函数选择不可靠比特进行路径分裂，并通过辅助路径度量 (Auxiliary Path Metric, APM) 值对不可靠路径进行剪枝，以此达到减少译码复杂度的效果。最后通过仿真实验验证了 PSS-RF-SCL 译码算法能够在不影响性能的情况下，降低译码复杂度。

第四章：候选路径选择策略辅助并行分裂简化译码算法的研究。该章是对 SCL 译码算法的计算端进行优化。首先介绍了针对特殊节点采用简化译码的 Fast-SCL 译码算法，介绍了其中的各种特殊节点及其对应的译码流程。并分析了在 Fast-SCL 译码算法的 Rate-1 节点中增加其译码时延的主要因素。接着提出了一种新的采用候选路径筛选的并行分裂方式，该方式可通过路径的并行分裂降低 Rate-1 节点的译码时间步长，并且新提出的候选路径筛选方式降低候选路径数量的效果较好，可以抑制并行分裂带来的候选路径数过多的问题。利用该方式提出了 PS-PSSCL (Path Selection strategy assisted Parallel Split and Simplify SCL) 译码算法。最后通过仿真实验得出，PS-PSSCL 译码算法相较于 Fast-SCL 译码算法能够在不影响性能的情况下降低 Rate-1 节点的译码时间步长，并且候选路径数量相较于其它并行分裂译码算法更低，能更灵活的适应各种传输环境。

第五章：总结与展望。对本文的研究内容与主要创新工作进行总结，并分析本文研究工作存在的不足，表明进一步的研究方向。

第2章 极化码基础理论分析

2.1 本章引言

在第一章介绍了极化码的研究现状后，本章主要对极化码的基础理论进行分析阐述。本章首先介绍极化码通信的重要基础——信道极化理论。然后介绍在编码阶段对性能有着重要影响的信道可靠性估计方法，接着描述整个编码过程。最后介绍两种基础的译码算法。

2.2 信道极化理论

信道极化是极化码构造的核心理论基础。其本质在于将多个独立的 B-DMC 信道通过特定的合并与拆分操作，实现信道容量的两极分化现象。具体而言，首先对 N 个相同的原始信道进行递归式合并，形成一个综合性的矢量信道；随后通过对合并信道的拆分处理，获得 N 个具有相关性的比特子信道。随着码长 N 的增加，这些子信道的对称容量将呈现出明显的极化趋势：一部分子信道的容量趋近于 1，适合承载信息比特；另一部分子信道的容量趋近于 0，适合传输固定的冻结比特。这种信道容量的极化效应为实现逼近信道容量的编码方案提供了理论依据。

2.2.1 信道的参数

为了量化上述的信道极化过程，需要两个关键指标来评估信道的特性。以 W 表示 B-DMC 信道。输入是 x ，输出是 y ， $W(y|x)$ 表示信道转移概率。这两个重要的指标分别为信道容量 $I(W)$ 和巴氏参数 $Z(W)$ 。 $I(W)$ 的计算公式如式(2-1)所示。

$$I(W) = \sum_{y \in Y} \sum_{x \in X} \frac{1}{2} W(y|x) \log_2 \frac{W(y|x)}{\frac{1}{2} W(y|0) + \frac{1}{2} W(y|1)} \quad (2-1)$$

式中 $I(W)$ 表示在特定信道条件下，系统能够达到的最大无误传输速率。其取值范围为 $[0,1]$ ，容量越高，信道越可靠。 $Z(W)$ 的计算公式如式(2-2)所示。

$$Z(W) = \sum_{y \in Y} \sqrt{W(y|0)W(y|1)} \quad (2-2)$$

式中 $Z(W)$ 定义为最大似然判决错误概率的上界，用于衡量信道的“噪声程度”。其取值范围同样为 $[0,1]$ 。 $Z(W)$ 越接近 0，信道越纯净；越接近 1，信道越嘈杂。同时两者之间存在严格的此消彼长关系，可以由式(2-3)所示的不等式描述。

$$\log_2 \frac{2}{1+Z(W)} \leq I(W) \leq \sqrt{1-Z(W)} \quad (2-3)$$

当 $Z(W) \rightarrow 0$ 时, 必有 $I(W) \rightarrow 1$ 。当 $Z(W) \rightarrow 1$ 时, 必有 $I(W) \rightarrow 0$ 。信道极化的过程, 实质就是推动子信道的 $I(W)$ 和 $Z(W)$ 向 $(0,1)$ 和 $(1,0)$ 这两个角落聚集的过程。

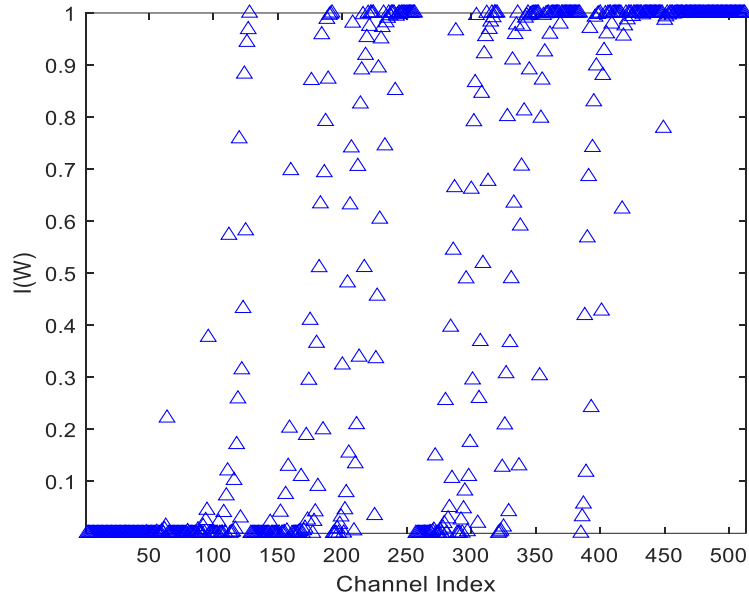


图 2-1 码长 512 的极化码信道容量分布

Fig.2-1 Channel capacity distribution of polar codes with code length of 512

如图 2-1 所示是码长为 512 的极化码在极化后的信道容量。从图 2-1 中可看出大部分信道的信道容量已经分化为 0 和 1 两个极端, 但仍存在处于中间状态的信道。这种中间状态的存在, 说明信道并未完全极化。只有两级分化的信道占比为 100% 时, 信道才是彻底完成极化。这种两级分化的比例跟码长有主要关系, 当码长 $N \rightarrow \infty$ 时, 将不存在处于中间状态的信道, 极化比例将达到 100%。容量接近 1 的子信道比例等于原信道容量 $I(W)$, 容量接近 0 的子信道比例等于 $1-I(W)$ 。这意味着, 极化过程没有损失任何信道容量, 只是将容量“重新分配”到了不同的子信道上。

信道极化由信道合并和信道分裂两部分组成, 接着将从这两个方面进行详细地说明。

2.2.2 信道合并

在传统的通信模型中, 如果我们拥有 N 个独立的 B-DMC 信道, 最直观的做法是将 N 个信息比特直接一对一地送入这 N 个信道。这种模式下, 每个比特的命运完全取决于它所在的那个特定信道的噪声状况, 比特之间互不干涉。信道合并颠覆

了这一情况，它的本质不是在物理层面上连接信道，而是执行了一次编码。通过这一操作，原本独立的输入比特 $(u_1, u_2, \dots, u_N)$ “搅拌在一起” 生成了新的发送序列 $(x_1, x_2, \dots, x_N)$ 。这种比特耦合机制是产生信道极化现象的根本原因。

N 个独立信道相互耦合生成的合并信道为 $W_N: X^N \rightarrow Y^N$, $N = 2^n$, $n \geq 0$ 。当 $N=2$ 时，两个独立的信道将合并生成 $W_2: X^2 \rightarrow Y^2$ ，其过程如图 2-2 所示。

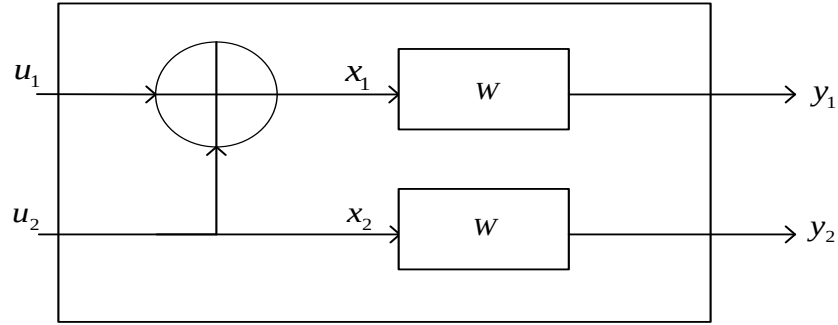


图 2-2 信道 W_2 示意图

Fig.2-2 Schematic diagram of channel W_2

图 2-2 中, $x_1 = u_1 \oplus u_2$, $x_2 = u_2$ 。信道 W_2 的信道转移概率可通过公式(2-4)计算。

$$W_2(y_1, y_2 | u_1, u_2) = W(y_1 | u_1 \oplus u_2)W(y_2 | u_2) \quad (2-4)$$

当 $N=4$ 时，一共有两级合并，将已经合并的信道 W_2 看作一个整体，将两个独立的 W_2 合并成 $W_4: X^4 \rightarrow Y^4$ 。其过程如图 2-3 所示。

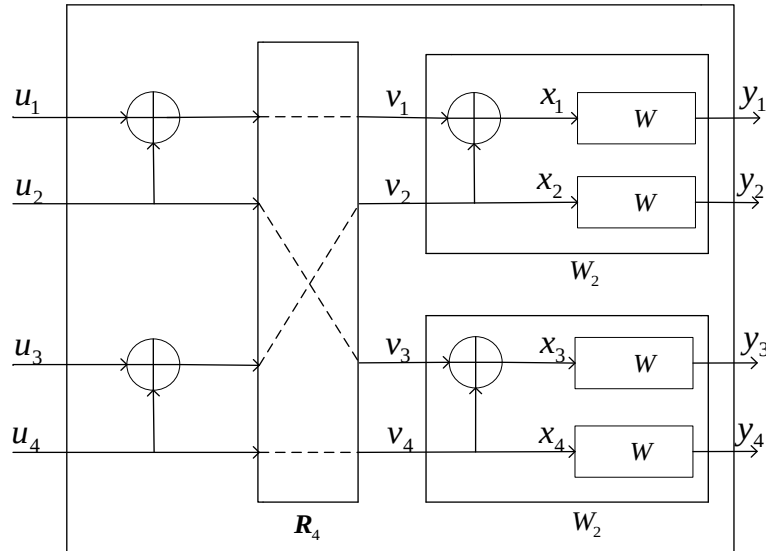


图 2-3 信道 W_4 示意图

Fig.2-3 Schematic diagram of channel W_4

图 2-3 中, R_4 是比特置换操作, 其作用是将 (s_1, s_2, s_3, s_4) 映射成 (s_1, s_3, s_2, s_4) 。这种置换操作的作用是让译码端译出的结果保持与输入端的顺序一致性。

信道 W_4 的转移概率可以写作式(2-5):

$$W_4(y_1^4 | u_1^4) = W_2(y_1^2 | u_1 \oplus u_2, u_3 \oplus u_4) W_2(y_3^4 | u_2, u_4) \quad (2-5)$$

基于上述规律进行 n 级递归, 两个独立的 $W_{N/2}$ 合并成 $W_N: X^N \rightarrow Y^N$ 。其过程如图 2-4 所示。

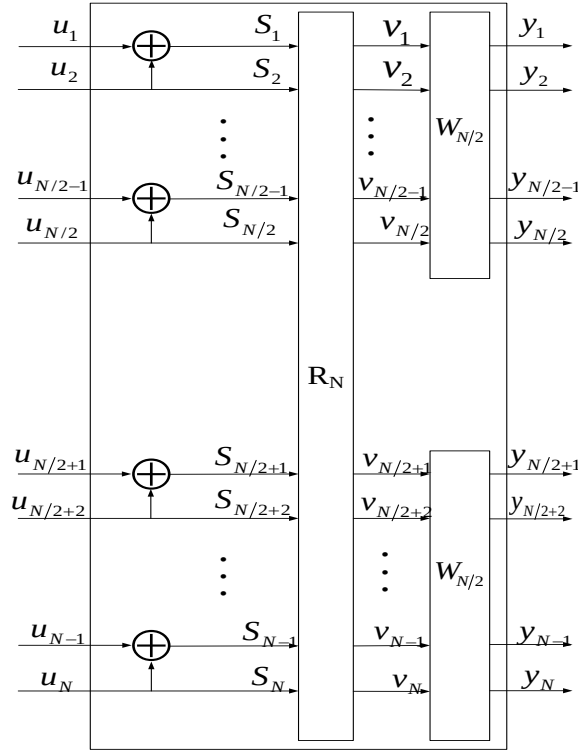


图 2-4 N 个信道合并示意图

Fig.2-4 N channels merging diagram

图 2-4 中输入的码字按照式(2-6)进行第一次耦合被转换为 s_1^N 。

$$\begin{cases} s_{2i-1} = u_{2i-1} \oplus u_{2i}, & 1 \leq i \leq N/2 \\ s_{2i} = u_{2i} \end{cases} \quad (2-6)$$

然后再经过 R_N 的置换操作将 $(s_1, s_2, s_3, \dots, s_N)$ 重排序为 $(s_1, s_3, \dots, s_{N-1}, s_2, s_4, \dots, s_N)$, 再通过一系列的耦合得到对应的极化码码字 $(v_1, v_2, v_3, \dots, v_N)$ 。将 u_1^N 与 v_1^N 的关系通过式(2-7)进行表达。

$$v_1^N = u_1^N G_N \quad (2-7)$$

式中, G_N 为生成矩阵, $G_N = B_N F^{\otimes n}$; B_N 是比特重排序矩阵; $F^{\otimes n}$ 是矩阵 F 的 n 次克罗内克积。 W_N 的信道转移概率如式(2-8)所示。

$$W_N(y_1^N | u_1^N) = W^N(y_1^N | u_1^N G_N) \quad (2-8)$$

2.2.3 信道分裂

如图 2-5 所示, 是联合信道 W_8 的信道分裂示意图。

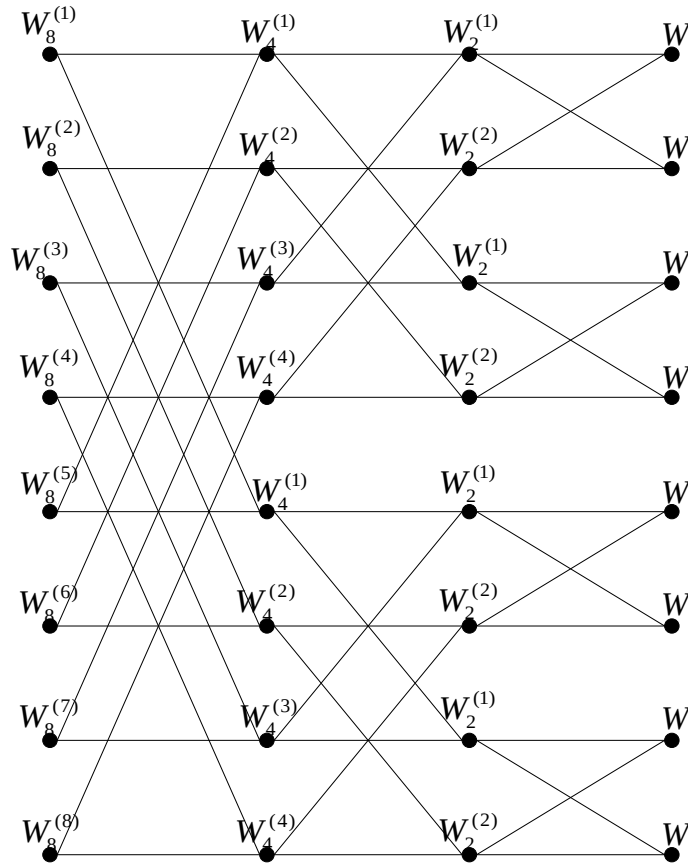


图 2-5 信道 W_8 的分裂示意图

Fig.2-5 Channel W_8 splitting diagram

信道分裂是信道极化过程中继信道合并后的第二部分。通过数学方法将 W_N 拆分成 N 个关联的信道 $W_N^{(i)}: X \rightarrow Y^N \times X^{i-1}$, $1 \leq i \leq N$ 。将第 i 个子信道的转移概率定义如公式(2-9)所示。

$$W_N^{(i)}(y_1^N, u_1^{i-1} | u_i) = \sum_{u_{i+1}^N \in X^{N-i}} \frac{1}{2^{N-i}} W_N(y_1^N | u_1^N) \quad (2-9)$$

式(2-9)中 (y_1^N, u_1^{i-1}) 表示子信道 $W_N^{(i)}$ 的输出； u_i 表示子信道 $W_N^{(i)}$ 的输入。先将联合信道 W_N 第一次分裂为 2 个 $W_{N/2}$ ，再将 2 个 $W_{N/2}$ 分裂为 4 个 $W_{N/4}$ ，以此类推，直到分裂形成 N 个极化子信道 W 。

分裂产生的序号为奇数的极化子信道的转移概率计算如式(2-10)所示。

$$W_N^{(2i-1)}(y_1^N, u_1^{2i-2} | u_{2i-1}) = \frac{1}{2} \sum_{u_{2i}} [W_{N/2}^{(i)}(y_1^{N/2}, u_{1,o}^{2i-2} \oplus u_{1,e}^{2i-2} | u_{2i-1} \oplus u_{2i}) \cdot W_{N/2}^{(i)}(y_{N/2+1}^N, u_{1,e}^{2i-2} | u_{2i})] \quad (2-10)$$

分裂产生的序号为偶数的极化子信道的转移概率计算如式(2-11)所示。

$$W_N^{(2i)}(y_1^N, u_1^{2i-1} | u_{2i}) = \frac{1}{2} W_{N/2}^{(i)}(y_1^{N/2}, u_{1,o}^{2i-2} \oplus u_{1,e}^{2i-2} | u_{2i-1} \oplus u_{2i}) \cdot W_{N/2}^{(i)}(y_{N/2+1}^N, u_{1,e}^{2i-2} | u_{2i}) \quad (2-11)$$

式(2-10)和式(2-11)中 $u_{1,o}^{2i-2}$ 表示序列 u_1^{2i-2} 中序号为奇数的比特序列； $u_{1,e}^{2i-2}$ 表示序列 u_1^{2i-2} 中序号为偶数的比特序列。

2.3 极化码编码

极化码编码的最重要部分就是选取可靠性高的信道。利用可靠信道传输信息比特，利用不可靠信道传输冻结比特。不同的信道可靠性估计方法，都有不同的侧重点和性能。

2.3.1 极化子信道的可靠性估计

目前存在的主流可靠性估计方法主要有巴氏参数法、GA 法和极化权重法。下面将对其进行介绍。

方法一：巴氏参数法。巴氏参数主要针对 BEC 信道，设信道 W 为 BEC 信道， $Z(W_N^{(i)})$ 的计算公式如式(2-12)，式(2-13)所示。

$$Z(W_N^{(2i-1)}) = 2Z(W_{N/2}^{(i)}) - Z(W_{N/2}^{(i)})^2 \quad (2-12)$$

$$Z(W_N^{(2i)}) = Z(W_{N/2}^{(i)})^2 \quad (2-13)$$

巴氏参数的值越大，表明译码的错误概率上限越高，该信道就越不可靠。因此，需要选择巴氏参数较小的极化子信道进行传输。

但在非 BEC 信道下，极化子信道的 $Z(W_N^{(i)})$ 计算如式(2-14)所示。

$$Z(W_N^{(2i-1)}) \leq 2Z(W_{N/2}^{(i)}) - Z(W_{N/2}^{(i)})^2 \quad (2-14)$$

由式(2-14)可知, 非 BEC 信道的 $Z(W_N^{(i)})$ 只能计算边界值, 所以巴氏参数法只针对 BEC 信道具有极高的实用价值, 在非 BEC 信道下则会受限。

方法二: GA 法。GA 法是用来在 AWGN 信道中构造 LDPC 码的, 并且是密度进化法的简化处理。GA 法在极化码中也非常适用, 该方法在保证性能的同时实现了高效计算, 已成为 AWGN 信道下极化码构造的标准方法, 为极化码在实际通信系统中的应用奠定了基础。

当信道为 AWGN 信道, 极化码码长为 N , 使用的调制方式为二进制相移键控 (Binary Phase Shift Keying, BPSK) 调制, 接收信号可表示为 $y_i = x_i + n_i, i = 1, 2, \dots, N$ 。其中 $x_i \in (-1, 1)$ 为调制后的发送符号。 n_i 为噪声, 服从均值为 0、方差为 σ^2 的高斯分布, 即 $n_i \sim N(0, \sigma^2)$ 。当 $x_i = 1$ 时, $y \sim N(1, \sigma^2)$, 当 $x_i = -1$ 时, $y \sim N(-1, \sigma^2)$ 。当发送端发送全 0 码字, 此时所有 x_i 都为 1。接收端的对数似然比如式(2-15)所示。

$$\text{LLR}(y_i) = \ln \frac{p(y_i | x_i = 1)}{p(y_i | x_i = -1)} = \frac{2}{\sigma^2} y_i \quad (2-15)$$

由式(2-15)可知 $\text{LLR}(y_i)$ 服从高斯分布其均值为 $2/\sigma^2$, 方差为 $4/\sigma^2$ 。方差为均值的两倍, 这一性质被称为对称条件, 是高斯近似理论的基础。因为极化的过程表示为 $(W_N^{(i)}, W_N^{(i)}) \rightarrow (W_{2N}^{(2i-1)}, W_{2N}^{(2i)})$, 因此只需要通过 $W_N^{(i)}$ 的 LLR 均值计算出 $W_{2N}^{(2i-1)}$ 和 $W_{2N}^{(2i)}$ 的 LLR 均值即可。 $W_{2N}^{(2i-1)}$ 的 LLR 均值计算如公式(2-16)所示。

$$\begin{aligned} L_{2N}^{(2i-1)}(y_1^{2N}, u_1^{2i-2}) &= \ln \frac{W_{2N}^{(2i-1)}(y_1^{2N}, u_1^{2i-2} | u_{2i-1} = 0)}{W_{2N}^{(2i-1)}(y_1^{2N}, u_1^{2i-2} | u_{2i-1} = 1)} \\ &\stackrel{(a)}{=} \ln \frac{\sum_{u_{2i}} W_N^{(i)}(y_1^N, u_{1,o}^{2i-2} \oplus u_{1,e}^{2i-2} | u_{2i}) W_N^{(i)}(y_{N+1}^{2N}, u_{1,e}^{2i-2} | u_{2i})}{\sum_{u_{2i}} W_N^{(i)}(y_1^N, u_{1,o}^{2i-2} \oplus u_{1,e}^{2i-2} | u_{2i} \oplus 1) W_N^{(i)}(y_{N+1}^{2N}, u_{1,e}^{2i-2} | u_{2i})} \\ &\stackrel{(b)}{=} \ln \frac{e^{L_N^{(i)}(y_1^N, u_{1,o}^{2i-2} \oplus u_{1,e}^{2i-2}) + L_N^{(i)}(y_{N+1}^{2N}, u_{1,e}^{2i-2})} + 1}{e^{L_N^{(i)}(y_1^N, u_{1,o}^{2i-2} \oplus u_{1,e}^{2i-2})} + e^{L_N^{(i)}(y_{N+1}^{2N}, u_{1,e}^{2i-2})}} \\ &= 2 \tanh^{-1} \left(\tanh \frac{L_N^{(i)}(y_1^N, u_{1,o}^{2i-2} \oplus u_{1,e}^{2i-2})}{2} \tanh \frac{L_N^{(i)}(y_{N+1}^{2N}, u_{1,e}^{2i-2})}{2} \right) \end{aligned} \quad (2-16)$$

令 $L_N^{(i)}(y_1^N, u_{1,o}^{2i-2} \oplus u_{1,e}^{2i-2})$, $L_N^{(i)}(y_{N+1}^{2N}, u_{1,e}^{2i-2})$, $L_{2N}^{(2i-1)}(y_1^{2N}, u_1^{2i-2})$ 分别用 a, b, c 来表示。因发送端为全 0 序列, 故公式(2-16)可简化为式(2-17)。

$$\tanh(z/2) = \tanh(x/2) \tanh(y/2) \quad (2-17)$$

$W_{2N}^{(2i)}$ 的 LLR 均值计算如公式(2-18)所示。

$$\begin{aligned} L_{2N}^{(2i)}(y_1^{2N}, u_1^{2i-1}) &= \ln \frac{W_{2N}^{(2i)}(y_1^{2N}, u_1^{2i-1} | u_{2i} = 0)}{W_{2N}^{(2i)}(y_1^{2N}, u_1^{2i-1} | u_{2i} = 1)} \\ &= (1 - 2u_1^{2i-1}) L_N^{(i)}(y_1^N, u_{1,o}^{2i-2} \oplus u_{1,e}^{2i-2}) + L_N^{(i)}(y_{N+1}^{2N}, u_{1,e}^{2i-2}) \end{aligned} \quad (2-18)$$

式中，假设信源序列全是 0，所以 u_{2i} 是 0，令 $s = L_{2N}^{(2i)}(y_1^{2N}, u_1^{2i-1})$ ，有式(2-19)。

$$s = x + y \quad (2-19)$$

因为 x, y 是独立同分布的，所以统一用 x 的参数。并对式(2-17)和式(2-19)等号两边取均值如式(2-20)和(2-21)所示。

$$E\{\tanh(z/2)\} = E\{\tanh(x/2)\}^2 \quad (2-20)$$

$$E\{s\} = 2E\{x\} \quad (2-21)$$

进一步将公式(2-20)按照期望的定义展开得到公式(2-22)。

$$\int_{-\infty}^{+\infty} \frac{\tanh(z/2)e^{-\frac{(z-E\{z\})^2}{4E\{z\}}}}{\sqrt{4\pi E\{z\}}} dz = \left[\int_{-\infty}^{+\infty} \frac{\tanh(x/2)e^{-\frac{(x-E\{x\})^2}{4E\{x\}}}}{\sqrt{4\pi E\{x\}}} dx \right]^2 \quad (2-22)$$

令 $\varphi(t) = 1 - \int_{-\infty}^{+\infty} \frac{1}{\sqrt{4\pi t}} \tanh(\alpha/2) e^{-\frac{(\alpha-t)^2}{4t}} d\alpha$ ，则可将公式(2-22)进一步简化为公式(2-23)。

$$E\{z\} = \varphi^{-1}\{1 - [1 - \varphi(E\{x\})]^2\} \quad (2-23)$$

为了方便计算，通常将式(2-23)中的 φ 函数简化为式(2-24)进行计算。

$$\varphi(x) = \begin{cases} e^{-0.4527x^{0.86} + 0.0218}, & 0 < x < 10 \\ \sqrt{\frac{\pi}{x}} e^{-\frac{x}{4}} (1 - \frac{10}{7x}), & x \geq 10 \end{cases} \quad (2-24)$$

通过公式(2-21)，(2-23)能够递归得到每个极化子信道的 $L_N^{(i)}(y_1^N, \hat{u}_1^{i-1})$ 均值，均值越大表示极化子信道越可靠，因此高斯近似法就是通过极化子信道 $w_N^{(i)}$ 的 LLR 均值大小来估计信道可靠性。

方法三：极化权重法。极化权重法是继高斯近似法之后提出的一种低复杂度的极化码构造方法。其它构造方法需要考虑信道的情况，以及信噪比。而极化权重法是一种与信道无关的构造方案，仅通过计算每个比特信道的极化权重值来评估其可靠性。该方法在 5G 增强移动宽带场景的极化码构造中得到了广泛应用。

极化权重 $PW(i)$ 的计算公式定义为式(2-25)。

$$PW(i) = \sum_{j=0}^{n-1} b_j \beta^j \quad (2-25)$$

式中 i 是对应极化子信道的序号。 b_j 是序号 i 的二进制展开式中第 j 位的比特值， $\beta = \sqrt[4]{2}$ 。从公式(2-25)可以看出，极化权重的计算实质上是将索引 i 的二进制表示

视为一个以 β 为基的数值。子信道的极化权重越大，其信道可靠性越高。表 2-1 展示的是码长为 8 的极化权重。

表 2-1 码长为 8 的极化权重 PW(i)

Table2-1 PW(i) with a code length of 8

索引	极化权重 PW(i)	可靠性排名
0	0	8
1	1	7
2	1.1892	6
3	2.1892	4
4	1.4142	5
5	2.4142	3
6	2.6034	2
7	3.6034	1

2.3.2 极化码编码过程

在选取了可靠的极化子信道后就需要进行极化码码字的生成。极化码的编码基于一个生成矩阵 G_N 。极化码的编码过程可表示为线性变换，即将信息序列 $\mathbf{u}_1^N$ 转换为发送序列 $\mathbf{x}_1^N$ ，如式(2-26)所示。

$$\mathbf{x}_1^N = \mathbf{u}_1^N \mathbf{G}_N \quad (2-26)$$

前文已经提到 $G_N = B_N F^{\otimes n}$ 。这里详细讲解一下生成矩阵的构造。 B_N 的计算如式(2-27)所示。

$$B_N = R_N I_N \quad (2-27)$$

式中 R_N 为 2.2 节提到的置换矩阵， I_N 为 N 阶单位矩阵。

$F^{\otimes n}$ 表示对如式(2-28)所示的矩阵 F 进行 n 次克罗内克积。

$$\begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \quad (2-28)$$

克罗内克积的运算规则为：对于矩阵 $A = [a_{ij}]_{m \times n}$ 和矩阵 C ，其克罗内克积定义为式(2-29)。

$$A \otimes B = \begin{bmatrix} a_{11}C & a_{12}C & \cdots & a_{1n}C \\ a_{21}C & a_{22}C & \cdots & a_{2n}C \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}C & a_{m2}C & \cdots & a_{mn}C \end{bmatrix} \quad (2-29)$$

假设码长为 8，码率为 1/2，发送的信息为全 1 码字 $\{1,1,1,1\}$ 。根据极化子信道可靠性估计选取的信息比特位置为 $A \in \{4,6,7,8\}$ 。则混合冻结比特后的码字为 $\mathbf{u}_1^8 = (00010111)$ ，经过编码后的码字 $\mathbf{x}_1^8 = \mathbf{u}_1^8 F^{\otimes 3} = (01101001)$ 。

2.4 极化码译码

尽管在无限码长下，极化码已被严格证明能够达到香农极限。但是在实际情况中，码长往往无法达到理论所需的渐进长度。这一现实约束直接导致了信道极化不完全，从而在有限码长下产生了不可忽视的性能损失。因此，译码算法的选择成为了平衡系统性能与复杂度的关键博弈。本节主要介绍的是 SC 译码算法和 CA-SCL 译码算法这两种经典译码算法。

2.4.1 SC 译码算法

SC 译码算法是一种逐比特的译码算法，比特 u_i 的估计值会根据接收信号的值 y_1^N 以及已经估计出的序列 u_1^{i-1} 进行计算。方式是分别计算 $u_i = 0$ 和 $u_i = 1$ 时的信道转移概率 $w_N^{(i)}$ ，来获得该比特的 LLR。 u_i 的 LLR 定义如式(2-30)所示。

$$L_N^{(i)}(\mathbf{y}_1^N, \mathbf{u}_1^{i-1}) = \ln \frac{w_N^i(\mathbf{y}_1^N, \mathbf{u}_1^{i-1} | u_i = 0)}{w_N^i(\mathbf{y}_1^N, \mathbf{u}_1^{i-1} | u_i = 1)} \quad (2-30)$$

通过式(2-30)得到比特 u_i 的对数似然比后就可以利用式(2-31)来判决它的值。

$$u_i = \begin{cases} 0, i \in A^c \text{ 或 } L_N^{(i)}(\mathbf{y}_1^N, \mathbf{u}_1^{i-1}) \geq 0 \\ 1, i \in A \text{ 且 } L_N^{(i)}(\mathbf{y}_1^N, \mathbf{u}_1^{i-1}) < 0 \end{cases} \quad (2-31)$$

式中 A^c 表示冻结比特的集合， A 表示信息比特的集合。

通过式(2-31)获得 u_i 的值后，又需要利用该值去计算下一个比特的 LLR。但(2-30)属于定义式，没有实际计算的操作性，LLR 可以递归计算，如式(2-32)和(2-33)所示。

$$\begin{aligned} & L_N^{(2i-1)}(\mathbf{y}_1^N, \hat{\mathbf{u}}_1^{2i-2}) \\ &= 2 \tanh^{-1} \left(\tanh \left(L_{N/2}^{(i)}(\mathbf{y}_1^{N/2}, \hat{\mathbf{u}}_{1,o}^{2i-2} \oplus \hat{\mathbf{u}}_{1,e}^{2i-2}) / 2 \right) \cdot \tanh \left(L_{N/2}^{(i)}(\mathbf{y}_{N/2+1}^N, \hat{\mathbf{u}}_{1,e}^{2i-2}) / 2 \right) \right) \end{aligned} \quad (2-32)$$

$$L_N^{(2i)}(\mathbf{y}_1^N, \hat{\mathbf{u}}_1^{2i-1}) = L_{N/2}^{(i)}(\mathbf{y}_{N/2+1}^N, \hat{\mathbf{u}}_{1,e}^{2i-2}) + (-1)^{\hat{u}_{2i-1}} L_{N/2}^{(i)}(\mathbf{y}_1^{N/2}, \hat{\mathbf{u}}_{1,o}^{2i-2} \oplus \hat{\mathbf{u}}_{1,e}^{2i-2}) \quad (2-33)$$

式(2-32)是奇数序号子信道的 LLR 计算公式，式(2-33)是偶数序号子信道的 LLR 计算公式。它们都是通过迭代计算完成的，直到迭代到 $L_1^{(1)}$ 时。这时的 $L_1^{(1)}$ 就是真实信道的对数似然比，其值通过信道模型，和接收到的码字进行计算。实际中可通过高斯近似进行计算，如式(2-34)所示。

$$L_1^{(1)} = \frac{2y}{\sigma^2} \quad (2-34)$$

式(2-34)中, y 为接收到的值, σ^2 为噪声方差。

这里以一个码长为 4, 含有 3 个信息比特的极化码 SC 译码作为例子。如图 2-6 所示, 发送端的 u_1 为冻结比特 0, 其它信息比特也为 0 值。SC 译码前接收端真实信道的对数似然比为 $L_1^{(1)}(y_j)$ 。

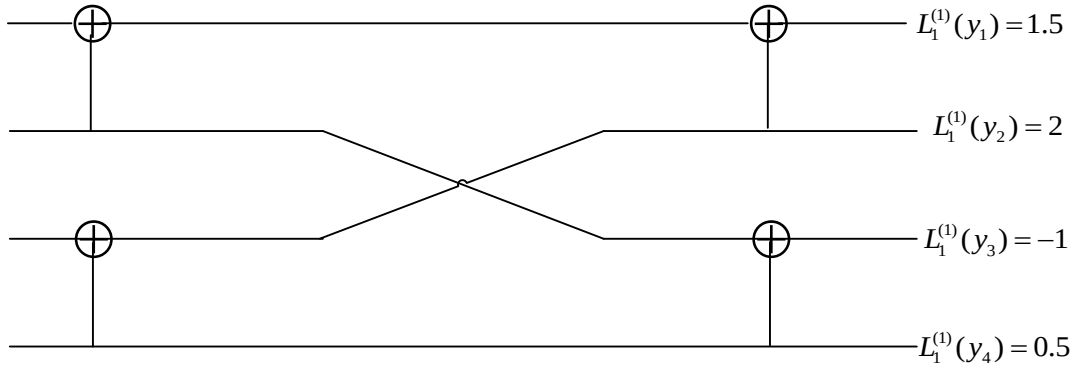


图 2-6 SC 译码前

Fig.2-6 Before SC decoding

在译码时, 需要从右向左递归, 通过式(2-32)和式(2-33)依次算出中间节点和发送端的对数似然比, 然后再根据发送端的对数似然比估计出码字。如图 2-7 所示。

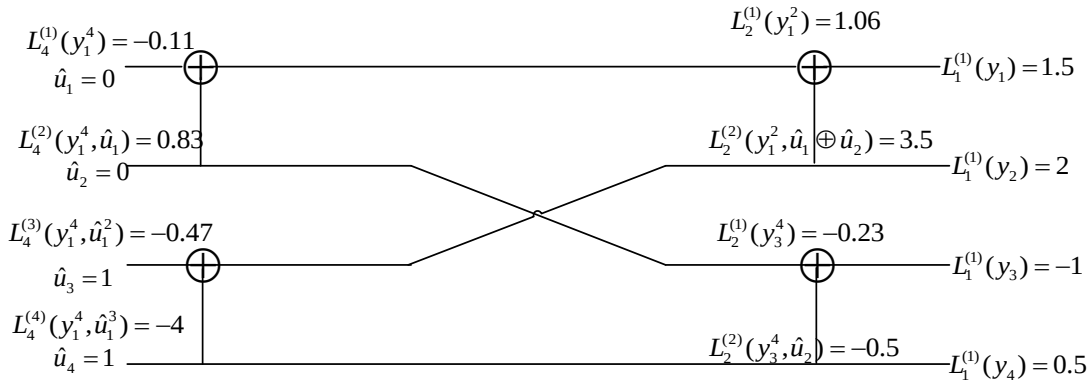


图 2-7 SC 译码后

Fig.2-7 After SC decoding

图 2-7 中, $\hat{u}_1$ 为冻结比特, 所以其值直接硬判决为 0。而其余位置则需要根据它们的 LLR 值来进行判决。其中 $\hat{u}_2$ 的值判决正确, 而 $\hat{u}_3$ 和 $\hat{u}_4$ 的判决值为 1 发生了译码错误。

2.4.2 CA-SCL 译码算法

SC 译码算法在中短码长下译码性能并不理想，并且具有错误传播现象：一旦某比特判决错误，该错误将影响后续所有比特的译码，导致错误扩散。针对该缺点，SCL 译码算法提出同时保留多条译码路径，尽可能的保留正确路径，防止因一个比特的对数似然比错误导致整个译码失败。为保留多条译码路径，SCL 译码算法提出在信息比特处进行路径分裂，即在现有路径上，同时分裂出该比特 0 和 1 的译码结果。同时 SCL 译码算法还会进行路径筛选，使得在完成一个比特的译码后，路径数不会超过 L 条。筛选的方法为，在完成一个比特的路径分裂后，计算每条分裂路径的 PM 值，保留其中 PM 值最小的 L 条路径进行后续的译码。在冻结比特处，SCL 译码算法不会进行路径分裂，而是直接判决为冻结比特 0，但仍然会进行 PM 值的积累。

PM 值的计算如式(2-35)所示。

$$PM_l^{(i)} = \begin{cases} PM_l^{(i-1)}, & \text{if } u_{i,l} = \frac{1}{2}[1 - \text{sign}(L_N^{(i)}(y_1^N, u_1^{i-1}))] \\ PM_l^{(i-1)} + |L_N^{(i)}(y_1^N, u_1^{i-1})|, & \text{else} \end{cases} \quad (2-35)$$

式中符合 LLR 判决的 PM 值将保持不变，而不符合 LLR 判决的 PM 值将会被累计惩罚值。PM 值越大的路径越不可靠。

图 2-8 是极化码 SCL 译码算法的译码树。一共 4 个比特， u_1 为冻结比特，其余位置为信息比特，最大保留路径数 $L=4$ 。

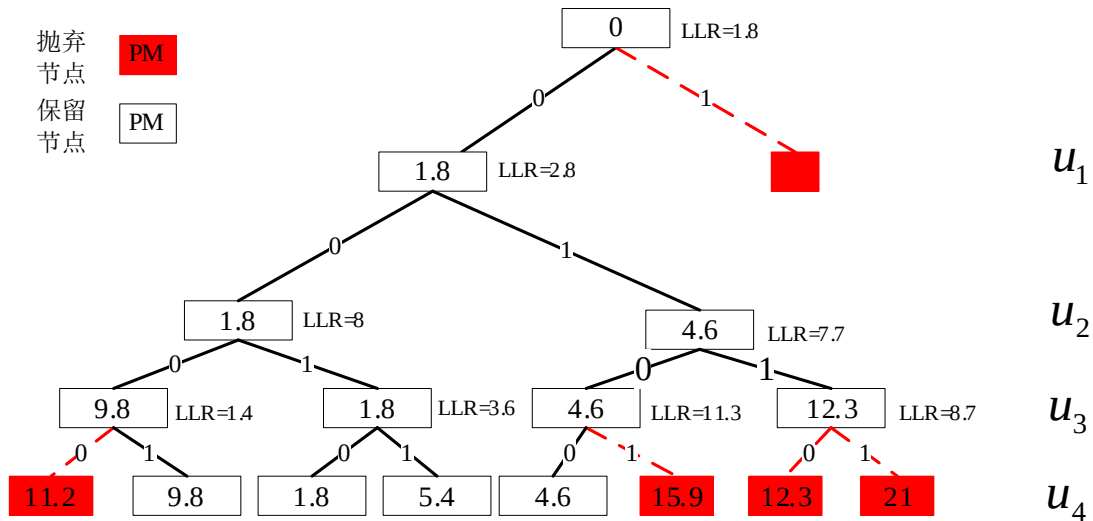


图 2-8 SCL 译码树

Fig.2-8 SCL decoding tree

图 2-8 中, u_1 为冻结比特, 所以不需要参考 LLR 直接判决为 0, 但需要累计 PM 值。在 u_2 和 u_3 处分别进行路径分裂得到了 4 条译码路径。在 u_4 处进行路径分裂产生了 8 条分裂路径, 因此通过 PM 值排序只保留其中 PM 值最小的 4 条。图 2-9 是 CA-SCL 译码算法的流程图。

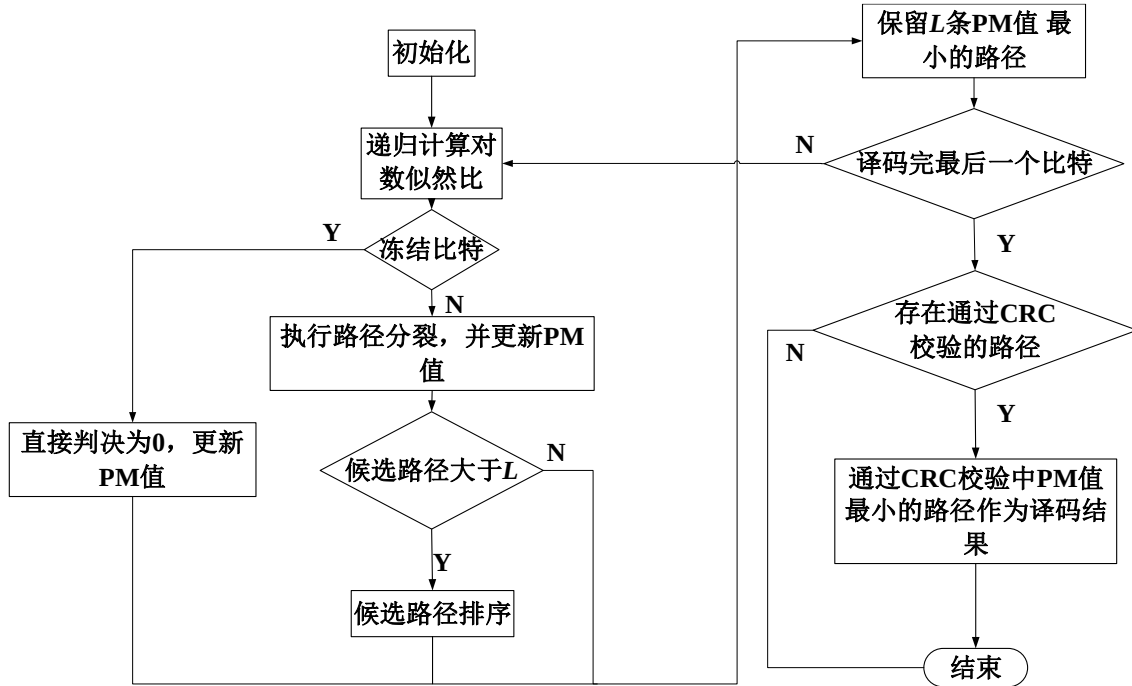


图 2-9 CA-SCL 译码算法流程图

Fig.2-9 The flowchart of CA-SCL decoding algorithm

在 SCL 译码算法中, 虽然通过路径搜索能够尽可能地保留候选路径, 但正确路径仍有可能因在译码过程中某一位受到较大的惩罚值, 导致其路径度量值相对较高, 进而使其在 L 条候选路径中排名靠后, 甚至被完全排除在保留列表之外。同时, 错误路径也可能因累积度量较低而进入最终的候选集合。这两个因素均是导致 SCL 译码算法性能下降的重要原因。为应对上述问题, 文献[11]提出了 CA-SCL 译码算法, 通过在编码端添加 CRC 校验序列, 并在 SCL 译码结束后对保留的 L 条路径依次进行 CRC 校验, 从而有效识别并剔除未能通过校验的错误路径。最终, 译码器从所有通过 CRC 校验的候选路径中选择 PM 值最小的路径作为最终的译码输出。借助 CRC 的校验能力, CA-SCL 译码算法能够显著提升正确路径的识别概率, 从而有效改善 SCL 译码算法在高信噪比区域的错误问题, 整体上大幅提升极化码的译码性能。CA-SCL 译码算法的提出使极化码的性能可以与 LDPC 码和 Turbo 码相媲美。

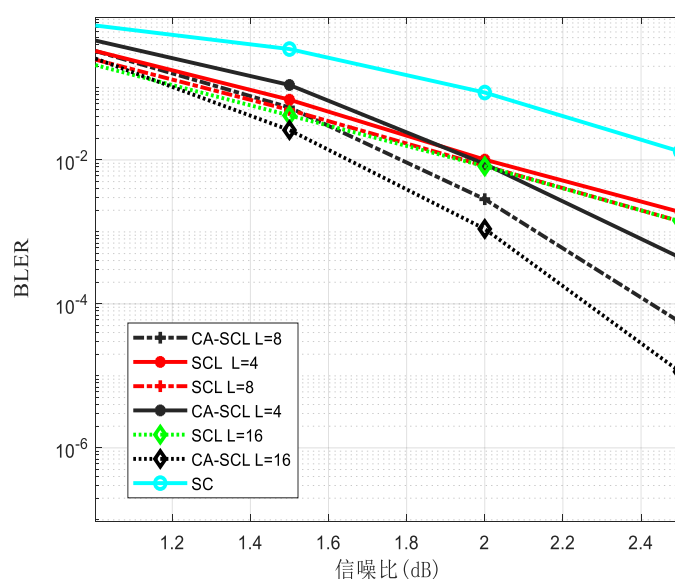


图 2-10 经典译码算法的误块率仿真对比图

Fig. 2-10 Simulation comparison diagram of block error rate for classic decoding algorithms

如图 2-10 所示, 是 SC、SCL 与 CA-SCL 这几种经典译码算法在加性高斯白噪声信道下的误块率性能对比曲线。其码长都设置为 1024, 码率为 1/2。同时, 为了说明列表数 L 对算法性能的影响, 对 SCL 译码算法和 CA-SCL 译码算法采用了不同列表数进行仿真。从图 2-10 中可看出, 使用了译码列表的 SCL 译码算法相较于 SC 译码算法有了明显的性能提升。但在 SCL 译码算法当中, 单纯增加列表规模 L 所带来的性能改善呈现出明显的边际递减效应, 当 L 从 8 增加到 16 时, 译码性能曲线几乎完全重合。这说明在 SCL 算法内部存在制约其性能进一步提升的瓶颈。通过实验深入分析 PM 值变化规律可以发现, 限制 SCL 译码算法性能的关键点在于正确译码路径有较大可能性排在列表靠后的位置。这是由于极化码的编码结构导致某些信息比特的对数似然比绝对值偏小, 导致正确路径对应的度量值在排序过程中落后于若干竞争路径。单纯扩大列表规模虽然能够覆盖更多候选路径, 但仅依靠列表的广度难以确保正确路径在最终选择阶段被选出。因此利用 CRC 码较强的校验性能, 可以使排名靠后的正确路径也能被筛选出来, 以略微的复杂度损失获得了明显的性能提升。并且, CRC 码也提高了 SCL 译码算法的边际效应。在 CA-SCL 译码算法中, 列表数的增加对性能的提升非常明显。有效的列表规模上限达到了百万数量级。这明显增加了中短码长下极化码译码算法的实际性能上限, 使其获得了较强的灵活性与实用价值。

2.5 本章小结

本章主要围绕极化码的基础理论展开系统阐述。首先，详细介绍了由信道合并与信道分裂所构成的信道极化现象，深入分析了该理论对信息传输效率的深刻影响及其在实际通信系统中的潜在应用价值。在理解信道极化基本原理的基础上，进一步探讨了极化码编码阶段的关键任务，具体包括极化子信道的可靠性估计方法以及极化码码字的生成过程。其中，极化子信道可靠性的准确估计对于提升系统性能具有至关重要的作用，因此本章重点介绍了几种主流的可靠性估计方法。最后，系统阐述了极化码中经典的 SC 译码算法以及重要的 CA-SCL 译码算法，深入剖析了这两种基线译码算法的基本原理、实现流程及其在误码性能与复杂度之间的权衡关系，为在后续章节中极化码译码算法的研究奠定了理论与算法基础。

第3章 基于可靠性函数的极化码 PSS-RF-SCL 译码算法研究

3.1 本章引言

通过前一章介绍极化码的基本理论，了解了传统 CA-SCL 译码算法的流程。文献[33]提出的 PSS-SS-SCL 译码算法运用选择分裂路径的方法，减少了在 CA-SCL 译码算法当中不必要的路径分裂，但对于路径分裂比特的选择以及对不可靠路径的处理仍存在不足。本章将针对上述两方面，提出减少复杂度的优化策略。本章首先介绍 PSS-SS-SCL 译码算法的改进和译码流程，并分析其不足。针对 PSS-SS-SCL 译码算法的不足之处，提出降低复杂度的译码策略；通过提出的策略对 PSS-SS-SCL 译码算法进行优化，进而提出一种基于可靠性函数的极化码 PSS-RF-SCL 译码算法作为优化方案，PSS-RF-SCL 译码算法在几乎不降低性能的情况下进一步降低了 PSS-SS-SCL 译码算法的复杂度。最后进行仿真实验，验证所提出的算法能够在几乎不降低性能的情况下，在复杂度方面有所优化。

3.2 PSS-SS-SCL 译码算法

通过前一章对 SCL 译码算法的分析可知，传统的 SCL 译码算法在所有信息比特处都会进行路径分裂操作，以此来尽可能地保留正确译码路径。但 PSS-SS-SCL 译码算法的提出者则认为路径分裂操作并非对所有信息比特都是必要的，在那些可靠性高的信息比特处进行路径分裂并不会带来显著性能提升，反而是频繁的路径分裂造成了译码算法的时间复杂度和空间复杂度都明显上升。PSS-SS-SCL 译码算法的核心思想是通过区分信息比特的可靠性，将不可靠比特纳入搜索集，只对搜索集中的比特进行路径分裂操作，而对搜索集外的比特直接执行硬判决，尽可能地在降低性能的情况下，降低复杂度。PSS-SS-SCL 译码算法的译码流程如图 3-1 所示。

在深入分析 PSS-SS-SCL 译码算法的流程后可以看出，该算法的核心机制在于构建一个搜索集，通过仅对该搜索集中所包含的不可靠比特实施路径分裂操作，从而在保证译码性能的前提下降低算法复杂度。具体而言，搜索集的作用是对参与路径分裂的比特位置进行选择性的筛选，避免了在所有信息比特上进行冗余的路径扩展，进而有效控制了搜索树的规模与计算开销。然而，若在关键位置上未能包含实际影响译码性能的不可靠比特，则可能导致路径分裂的遗漏，从而对整体译码性能造成不利影响。由此可见，搜索集的构造质量决定了 PSS-SS-SCL 译码算法在译码

性能与计算复杂度之间的平衡效果。当前，PSS-SS-SCL 译码算法中的搜索集构造主要依据文献[55]中通过仿真实验所获得的结论，该文献基于大量实验数据对不同位置的比特可靠性进行了统计分析，提出了用于识别不可靠比特的经验性判据。

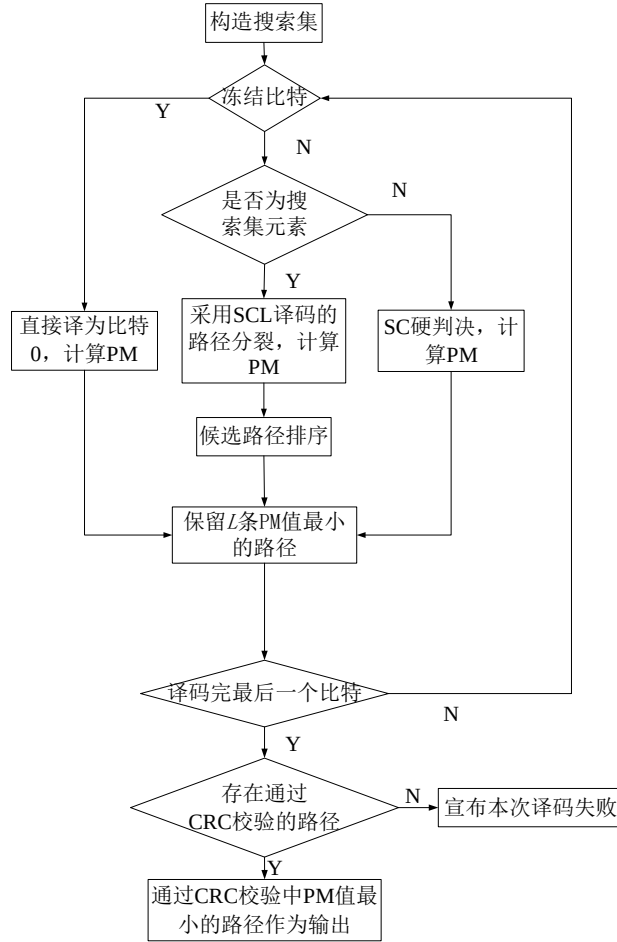


图 3-1 PSS-SS-SCL 译码算法的译码流程图

Fig. 3-1 The decoding flowchart of PSS-SS-SCL decoding algorithm

文献[55]的仿真实验主要是根据极化码译码二叉树的特殊节点来进行的。如图 3-2 所示，这是一个拥有 16 个比特的译码二叉树。

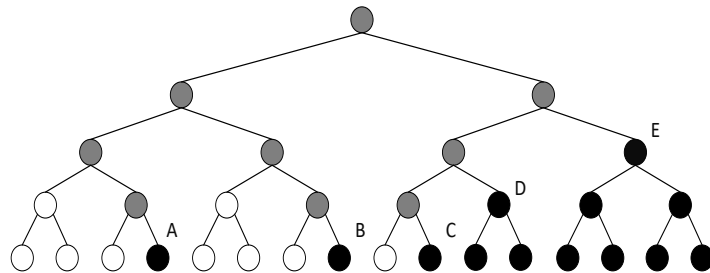


图 3-2 极化码译码二叉树示意图

Fig. 3-2 Polar code decoding binary tree diagram

图 3-2 中，白色节点表示其叶子节点全为冻结比特，黑色节点表示其叶子节点全为信息比特。灰色节点则表示其叶子节点中既有冻结比特，又有信息比特。像 $\{A, B, C, D, E\}$ 这样的特殊节点，则称为 Rate-1 节点。每个 Rate-1 节点可以看作一个只包含信息比特的极化码子块，虽然 Rate-1 节点全是传输信息比特的可靠信道，但其可靠性依然有差别。在长度为 $N = 2^n$ 的极化码结构中，极化信道的顺序标号为 W_N^i ， $0 \leq i \leq N-1$ 。若序号 p 的二进制展开比序号 q 的二进制展开刚好多一位 1，则存在公式(3-1)所示的关系。

$$\begin{cases} I(W_N^p) \geq I(W_N^q) \\ Z(W_N^p) \leq Z(W_N^q) \\ P_e^{ML}(W_N^p) \leq P_e^{ML}(W_N^q) \end{cases} \quad (3-1)$$

式(3-1)说明信道 p 从信道容量、巴氏参数、最大似然判决错误率三个方面都优于信道 q ，因此信道 p 是一个更好的信道。虽然这只是两个信道之间的偏序关系，但通过两两之间的比较可以得出 W_N^0 是 W_N^i ($0 \leq i \leq N-1$) 中最差的信道。因此每个 Rate-1 节点中的首个比特是该节点中最不可靠的，将其纳入搜索集。并且文献[56]通过仿真实验获得了结论，译码过程中首个发生错误的比特 99% 都会处于搜索集中。依靠搜索集对不可靠比特的识别，PSS-SS-SCL 译码算法在略微降低性能的情况下，有效地降低了译码复杂度。

这种通过搜索集来进行不可靠比特识别的方法，虽然能很好地捕获首个出现错误的比特，抑制错误传播现象。但因为其依据的是 Rate-1 节点中信道的偏序关系，并没有分析当前译码比特的实际情况，这也造成搜索集包含的不可靠比特数量较大，译码复杂度较高。并且 PSS-SS-SCL 译码算法只考虑了译码比特的可靠性，通过区分比特的可靠性从而采用不同的译码方法使得译码复杂度下降。但是在译码过程中没有考虑到不同路径之间也存在可靠性的差别，这使得许多不必要的译码路径继续占用译码资源，使得译码复杂度升高。

3.3 降低复杂度的译码策略

本节主要针对 PSS-SS-SCL 译码算法在识别不可靠比特和路径上的不足，提出两条针对性的改进译码策略。其中一条采用更符合译码实际情况的路径分裂策略，该策略通过计算可靠性函数来判断信息比特是否需要进行路径分裂。相较于现有的依靠搜索集来识别不可靠比特的方法，可靠性函数更贴合译码实际情况，识别不可靠比特更精准，可以减少路径分裂次数从而降低译码复杂度。另一条采用基于

APM 值的剪枝策略。通过 APM 值来区分路径的好坏，将被视作已经产生错误的路径剪枝处理，减少错误路径对后续译码情况的影响以及对译码资源的占用。

3.3.1 基于可靠性函数的路径分裂策略

在文献[28]中，为了更好地进行比特翻转，需要找到那些在首次译码过程中产生了译码错误的比特。因此其提出了一种用于评估任意一个信息比特在首次译码中是否出错的方式。如果在 $\hat{u}_i$ 处，一共分裂产生了 $2L$ 条路径，不失一般性地假设前 L 条路径被保留。那么可用公式(3-2)的值来评估该比特是否在首次译码中产生错误。

$$\log\left[\sum_{l=1}^L e^{-PM_l^{(i)}} / \left(\sum_{l=L+1}^{2L} e^{-PM_l^{(i)}}\right)^{1.2}\right] \quad (3-2)$$

式中 $e^{-PM_l^{(i)}}$ 是第 l 条译码路径 $\hat{u}_{i,l}$ 从 $\hat{u}_{i,1}$ 到 $\hat{u}_{i,l}$ 没有出现错误的概率的估计值。受此启发，提出一个可用于识别不可靠比特的可靠性函数。在译码过程中，译码出的码字序列为 $\hat{u}_1^N$ 。当 $\hat{u}_i$ 进行路径分裂时，现有的 $x(1 \leq x \leq L)$ 条译码路径将会分裂产生 $2x$ 条译码路径 $\hat{u}_{i,l}(1 \leq l \leq 2x)$ 。其中 x 条是符合 $\hat{u}_i$ 硬判决结果的路径，其 PM 值将保持不变。用 $\{A_1^{(i)}, A_2^{(i)}, \dots, A_x^{(i)}\}$ 来表示它们的 PM 值大小。另外 x 条是不符合 $\hat{u}_i$ 硬判决结果的路径，其 PM 值将会被添加惩罚而变大。用 $\{B_1^{(i)}, B_2^{(i)}, \dots, B_x^{(i)}\}$ 来表示它们的 PM 值大小。这些 PM 值可在路径分裂前直接计算得到。在式(3-2)的基础上进行改变得到可靠性函数如式(3-3)所示。

$$E(i) = \log\left[\sum_{l=1}^x e^{-A_l^{(i)}} / \left(\sum_{l=1}^x e^{-B_l^{(i)}}\right)^{1.2}\right] \quad (3-3)$$

可靠性函数由 x 条符合硬判决结果的路径的未错总概率与 x 条不符合硬判决结果的路径的未错总概率之比确定。该比值反映了在该比特位置上不同判决方向所对应路径的整体可靠性差异。当可靠性函数值较大时，意味着符合硬判决结果的路径在概率意义上优于不符合判决结果的路径，即由分裂所产生的备选路径的可靠性明显弱于原始路径，此时进行路径分裂的必要性较低，当前路径直接进行硬判决即可满足译码需求。反之，若可靠性函数值较小，则表明分裂所产生的路径在可靠性上已接近原始路径的水平，单纯依赖硬判决可能导致潜在的正确路径未能被保留下来，因此需要通过路径分裂将两类路径同时保留。

通过统计在不同码长和信噪比下，需要通过路径分裂来保留正确译码路径的（直接硬判决会出现错误的）比特的可靠性函数值，可得到一个稳定的范围。将该范围的上限设为阈值 α 。如表 3-1 所示，是统计出的不同码长和不同信噪比下阈值 α 的大小。在译码过程中，对于信息比特 u_i ，在执行路径分裂前，先计算其可靠性函

数值 $E(i)$ ，并与对应的阈值 α 进行比较。若 $E(i) < \alpha$ 才需要进行路径分裂，否则表示由分裂产生路径的可靠性明显低于原始路径，此时进行路径分裂的必要性较低。

表 3-1 阈值 α 的值Table3-1 The value of threshold α

码长	信噪比(dB)	α
1024	1	38
1024	1.5	40
1024	2	40
1024	2.5	43
1024	3	45
512	1	20
512	1.5	21
512	2	23
512	2.5	25
512	3	27

3.3.2 路径剪枝策略

虽然信息比特相较于冻结比特而言其可靠度更高，但由于码长的限制并没有实现完全极化，这导致信息比特之间也存在明显的可靠度差异。传统的 PSS-SS-SCL 译码算法正是考虑到这一点，所以对不同可靠度的比特采用不同的译码算法。但却忽略了路径之间的可靠度差异，对于明显不可靠的路径没有提前剪枝，依然让它们参与到后续的译码过程中，对后续译码产生影响。虽然在路径的分裂、排序、删除操作中有可能淘汰掉它们，但排序删除操作的复杂度较高，如果能通过提前剪枝的方式将其淘汰可以降低译码复杂度。

在 SCL 译码算法中通过 PM 值来衡量路径的可靠性。但 PM 值是在所有比特处累计惩罚值，导致其值的变化情况非常复杂。例如，正确译码路径的 PM 值可能由于在冻结比特处的累计导致其偏高。因此 PM 值的大小只适合用来进行路径间的相互比较，而不能直接通过数值大小判断该路径是否出现错误。在文献[36]中提出了一种改良后的 APM 值，因此本文在此基础上提出一种基于 APM 值的剪枝策略。

APM 值的计算如式(3-4)所示。

$$APM_l^{(i)} = \begin{cases} APM_l^{i-1} + (\hat{u}_i - \frac{1 - \text{sgn}(L_i[l])}{2})L_i[l] & i \in A^s \\ APM_l^{(i-1)} & \text{else} \end{cases} \quad (3-4)$$

式中 A^s 表示所有不可靠比特的集合，APM 只在不可靠比特处进行累计，而在冻结比特和其它信息比特处保持不变。因为不可靠比特是导致译码出现错误的关键位置，所以 APM 值越大，说明该路径在关键位上做出与信道信息相悖的决策次数越多，越可能是错误路径。

为基于 APM 值制定剪枝策略，要衡量路径 APM 值分布的特点。因此本文通过统计实验获得在译码过程中被剪枝的正确路径的 APM 值分布情况，如图 3-3 所示。

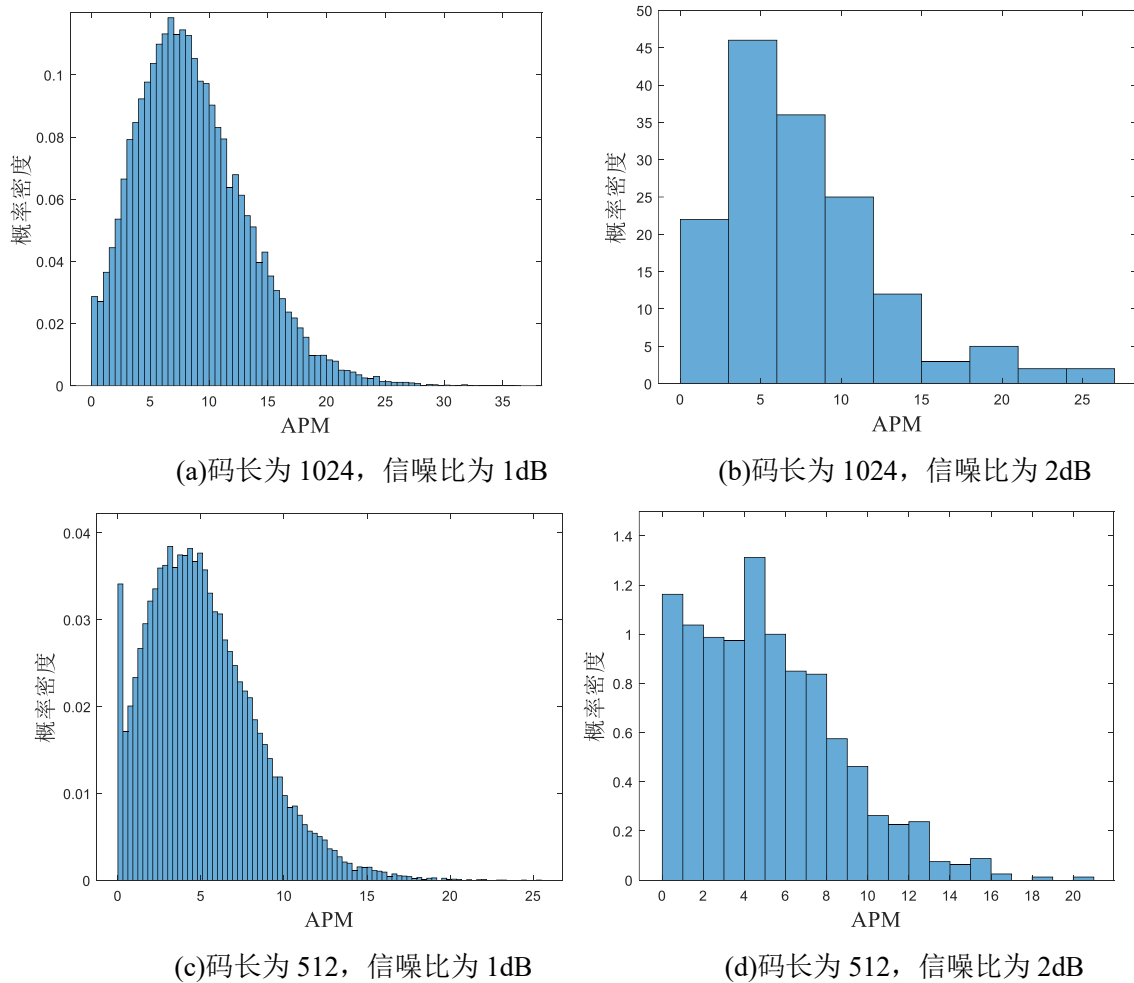


图 3-3 不同码长和信噪比下被淘汰的正确路径的 APM 值分布情况

Fig. 3-3 Distribution of APM values for eliminated correct paths under different code lengths and signal-to-noise ratios

如图 3-3 所示，即使是被淘汰的正确路径，其 APM 值依旧存在明确的上限。因此说明正确路径的 APM 值是相当稳定的，可以将超过一定上限的 APM 值路径视作错误路径，进行剪枝。

综上，提出的剪枝策略会首先设定一个阈值 β ，在每个信息比特译码完成后进行判断。若某条路径的 APM 值大于设定的阈值 β ，则该路径被剪枝。通过对明显产生错误的路径进行剪枝，减少了错误路径对后续译码的影响并且减少了多余路径的译码操作，使得译码复杂度降低。

表 3-2 阈值 β 的值Table3-2 The value of threshold β

码长	信噪比(dB)	β
1024	1	30
1024	1.5	30
1024	2	25
1024	2.5	21
1024	3	17
512	1	18
512	1.5	21
512	2	19
512	2.5	17
512	3	17

如表 3-2 所示，是通过大量统计实验得到的不同码长和信噪比下的阈值 β 的值。

3.4 一种基于可靠性函数的极化码 PSS-RF-SCL 译码算法

根据 3.3 节所提出的降低复杂度的译码策略，本节对 PSS-SS-SCL 译码算法进行优化与改进，提出一种基于可靠性函数的极化码 PSS-RF-SCL 译码算法。该算法的核心创新在于摒弃了原有依赖于搜索集的不可靠比特选取机制，转而采用式(3-3)所定义的可靠性函数作为路径分裂的量化判据。通过对每个信息比特的可靠性函数值进行计算与评估，PSS-RF-SCL 算法能够更加精准地识别出需要进行路径分裂的不可靠比特位置，从而在确保关键路径得以保留的前提下，有效减少了冗余的路径分裂操作次数，显著降低了译码过程的计算复杂度。此外，为进一步优化译码效率，该算法采用了基于 APM 的剪枝策略，即在每一个信息比特译码完成后，及时对当前路径列表中明显不可靠的路径进行剪枝处理。这一剪枝机制不仅能够及早剔除错误概率较高的路径，减少其对后续比特译码过程的干扰，同时也有助于控制路径列表的规模，避免路径数量的无谓增长。通过上述两方面的协同优化，PSS-RF-SCL 译码算法在维持良好译码性能的基础上，降低了译码复杂度。

3.4.1 PSS-RF-SCL 译码算法的译码过程

为降低译码复杂度，所提出的 PSS-RF-SCL 译码算法整体流程如图 3-4 所示。

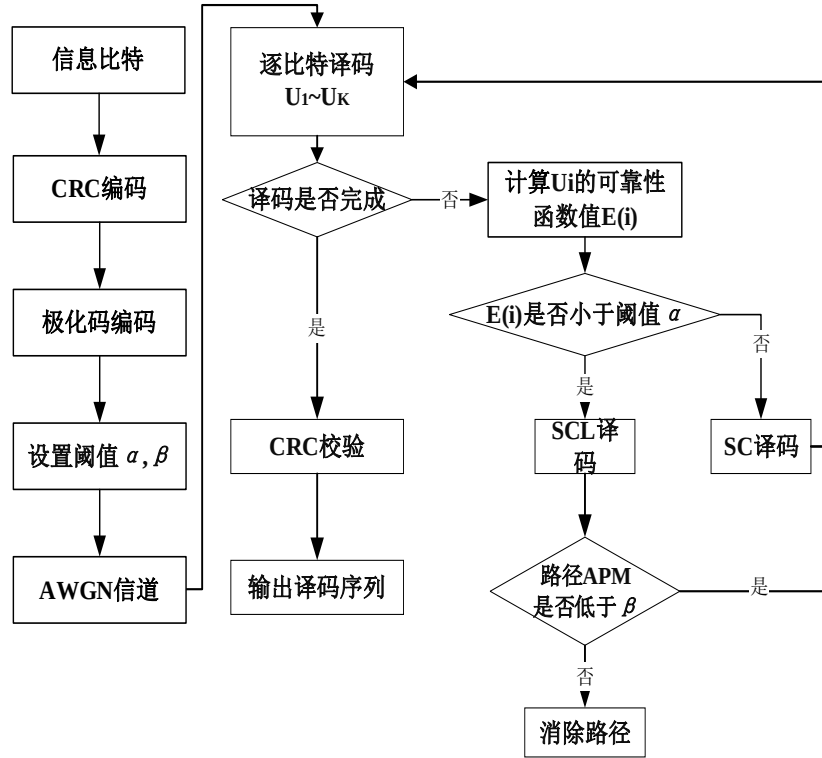


图 3-4 PSS-RF-SCL 译码算法的整体流程图

Fig. 3-4 Overall flowchart of PSS-RF-SCL decoding algorithm

基于上述设计思想，PSS-RF-SCL 译码算法根据可靠性函数、APM 值、阈值 α 和阈值 β ，实现了对译码路径分裂与剪枝的精细化控制。在进行路径分裂前，计算出符合硬判决结果的路径 PM 值 $A_x^{(i)}$ 和不符合硬判决结果的路径 PM 值 $B_x^{(i)}$ 。并依据式(3-3)所示的可靠性函数，由 $A_x^{(i)}$ 与 $B_x^{(i)}$ 计算出该比特的可靠性函数值 $E(i)$ 。随后，算法将 $E(i)$ 与预设阈值 α 进行比较。若 $E(i)$ 小于阈值 α ，表明当前比特位置上两类路径的可靠性差异较小，分裂产生的路径具有保留价值，因此执行完整的 SCL 译码以扩展路径。反之，若 $E(i)$ 大于阈值 α ，则说明符合硬判决结果的路径显著占优，分裂必要性较低，算法直接进行硬判决以降低复杂度。在每一信息比特译码完成后，算法进一步启动基于 APM 的剪枝机制，将当前路径列表中每条路径的 APM 值与预设阈值 β 进行比较，对 APM 值大于 β 的路径进行剪枝，从而及时清除可靠性较低的路径，避免其干扰后续比特的译码过程。通过上述阈值引导的分裂决策与剪枝操作的协同配合，PSS-RF-SCL 译码算法实现了对路径列表规模的有效控制，其具体译码流程与操作步骤如算法 3-1 所示。

算法 3-1 PSS-RF-SCL 译码算法

START**Input:** 码长 N , 接收端序列, 阈值 α , 阈值 β , 冻结比特集合**Output:** 译码估计序列

```

1: FOR  $i=1:N$ 
2: IF  $i \in A^C$ 
3:  $\hat{u}_i = 0$ 
4: ELSE
5: 计算出  $A_x^{(i)}$  和  $B_x^{(i)}$  的值
6: 计算出  $E(i)$  的值
7: IF  $E(i) < \alpha$ 
8: 执行 SCL 译码
9: ELSE
10: 执行 SC 译码
11: END IF
12: END IF
13: FOR  $l=1:L$ 
14: IF  $APM_l > \beta$ 
15: 消除该路径
16: END IF
17: END FOR
18: END FOR
19: IF 存在通过 CRC 校验路径
20: 选择其中 PM 值最小的路径作为输出序列  $\hat{u}_1^N$ 
21: ELSE
22: 选择所有路径中 PM 值最小的路径作为输出序列  $\hat{u}_1^N$ 
23: END IF

```

3.4.2 仿真分析

为了验证所提出的 PSS-RF-SCL 译码算法在复杂度方面的改善, 以及对性能的影响。本小节通过仿真实验对所提 PSS-RF-SCL 译码算法的误块率和复杂度与其它算法进行比较, 尤其是与现有的基于搜索集的路径分裂策略辅助 SCL 译码算法进行比较。在基于搜索集的算法中最经典的是 3.2 节所提到的 PSS-SS-SCL 译码

算法^[33]，其译码流程已经做过介绍。文献[36]所提出的 PSD-SCL 译码算法在此基础上更进一步，该方法通过提前识别并避免错误路径，从而减少路径分裂次数，并且同时提出了 SA(Statistical Approach)和 GA 两种识别错误路径的方式。为了公平的比较各个算法的性能，在仿真中对不同算法设置相同的码长 N ，码率 R ，最大列表数 L ，并且在各个算法中添加的校验码(CRC 码)的长度是相同的。各个算法均采用 $SNR = 2.5\text{dB}$ 的 GA 构造方法和 BPSK 调制，信道选用 AWGN 信道。

图 3-5 显示了在码长 $N=1024$ ， $R=1/2$ ，列表数 $L=8$ 时不同算法的误块率比较结果；图 3-6 显示了在码长 $N=1024$ ，码率 $R=1/2$ ，列表数 $L=16$ 时不同算法的误块率比较结果。根据图 3-5 和 3-6 可知在码长为 1024，码率为 1/2 时，在不同的列表数和信噪比下，本文提出的 PSS-RF-SCL 译码算法相较于 PSS-SS-SCL 译码算法^[33]，PSD-SCL GA 译码算法^[36]，性能几乎一致。并且跟图中所有基于搜索集的路径分裂策略辅助 SCL 译码算法相比，PSS-RF-SCL 译码算法相较于 CA-SCL 译码算法^[2]的性能损失最小。当列表数为 8 时，PSS-RF-SCL 在误块率为 10^{-5} 处仅损失 0.04dB。当列表数为 16 时，PSS-RF-SCL 在误块率为 10^{-4} 处仅损失 0.03dB，在图中的路径分裂选择算法中，其性能损失最低。虽然在 CA-SCL 译码算法的基础上进行路径分裂选择，会带来略微的性能损失，但其复杂度优化却是明显的。两幅图中 PSD-SCL SA 译码算法性能最差，是由于其更简单直接的路径分裂选择方式所造成的，这也使得其复杂度在传统的基于搜索集的路径分裂策略辅助 SCL 译码算法中是更低的。

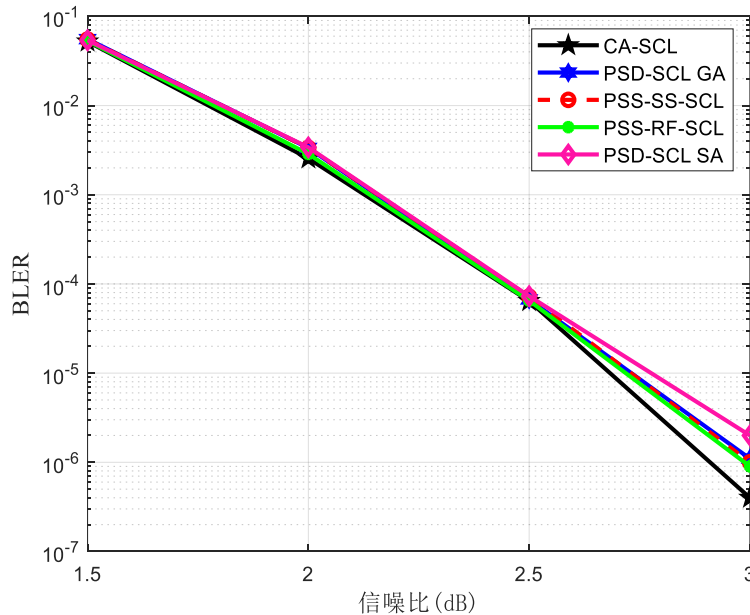
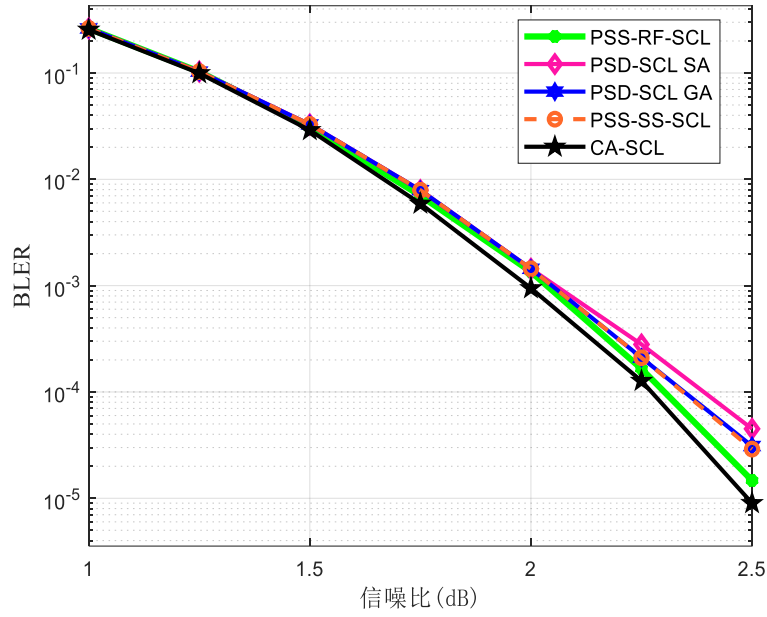
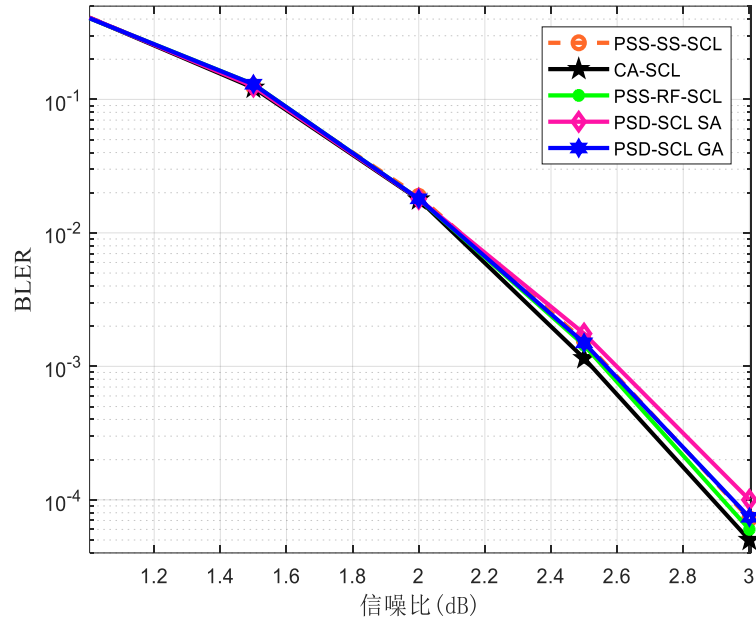


图 3-5 $N=1024, R=1/2, L=8$ 时各算法误块率仿真图

Fig. 3-5 Simulation graph of block error rate for each algorithm at $N=1024, R=1/2, L=8$

图 3-6 $N=1024, R=1/2, L=16$ 时各算法误块率仿真图Fig. 3-6 Simulation graph of block error rate for each algorithm at $N=1024, R=1/2, L=16$

从图 3-7 中可知在码长为 512，码率为 1/2 时，PSS-RF-SCL 译码算法的性能与其它基于搜索集的路径分裂策略辅助 SCL 译码算法相比没有任何下降，并且相较于 CA-SCL 译码算法的性能损失较小。由此可知，所提算法在不同码长下性能稳定，不会像 PSD-SCL SA 算法一样导致性能明显下降。

图 3-7 $N=512, R=1/2, L=8$ 时各算法误块率仿真图Fig. 3-7 Simulation graph of block error rate for each algorithm at $N=512, R=1/2, L=8$

经过上述表述,可知所提出的 PSS-RF-SCL 译码算法性能稳定。提出该算法的目的是在传统的路径分裂策略辅助 SCL 译码算法基础上进一步降低复杂度,因此接下来对复杂度进行比较。在 CA-SCL 译码算法中,使用的译码列表数越少,路径分裂次数越少,计算复杂度就越低,消耗的计算资源就越少。因此从上述两个方面来比较各个算法的复杂度。

在进行复杂度比较时,各个算法都设置码长 $N=1024$, 码率 $R=1/2$, 列表数 $L=8$, 添加 16 位校验码。图 3-8 比较了不同算法译码时的平均译码列表总数。

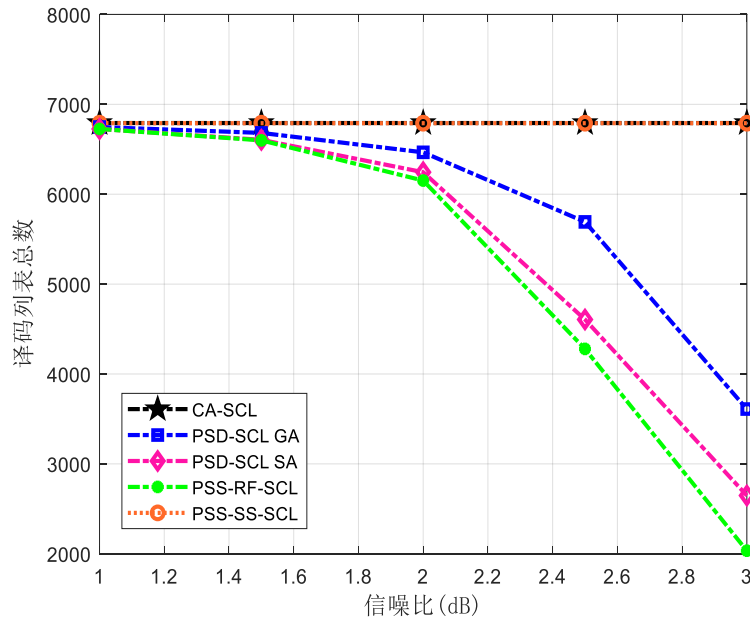


图 3-8 不同算法的平均译码列表总数

Fig. 3-8 Average total decoding list size for different algorithms

平均译码列表总数 L_{sum} 为译码每个比特使用的路径数总和, L_{sum} 越大译码复杂度越高。 L_{sum} 计算如式(3-5)所示。

$$L_{sum} = \sum_{i=1}^N L_i \quad (3-5)$$

式(3-5)中 L_i 表示第 i 个比特译码时译码器存活的路径数。

从图 3-8 可以看出相较于传统的 PSS-SS-SCL 和 CA-SCL 译码算法, PSS-RF-SCL 译码算法的译码列表总数在信噪比为 2.5dB 时下降了 37%, 在信噪比为 3dB 时下降了 70%。相较于复杂度更低的 PSD-SCL GA 译码算法, PSS-RF-SCL 译码算法的译码列表总数在信噪比为 3dB 时也下降了 44%。与判决更严格的 PSD-SCL SA 译码算法相比, 在信噪比为 3dB 时下降了 24%。因此看出所提出的 PSS-RF-SCL 译码算法在译码列表总数方面, 有着明显的优势。

图 3-9 比较的是不同算法的平均分裂次数，PSS-RF-SCL 译码算法的分裂次数在信噪比为 1dB 时为 80，相较于 CA-SCL 译码算法下降了 85%，相较于其它基于搜索集的路径分裂策略辅助 SCL 译码算法下降了 33%。在信噪比为 3dB 时，PSS-RF-SCL 译码算法的分裂次数为 8.3，相较于 PSD-SCL GA 译码算法下降了 45%。虽然 PSD-SCL SA 译码算法的分裂次数与所提算法在信噪比为 3dB 时基本一致，但是由于其简单直接的判断方式导致其误块率性能下降程度明显，性能损失较大。而 PSS-RF-SCL 译码算法不仅分裂次数下降程度更明显，而且译码性能也十分稳定。

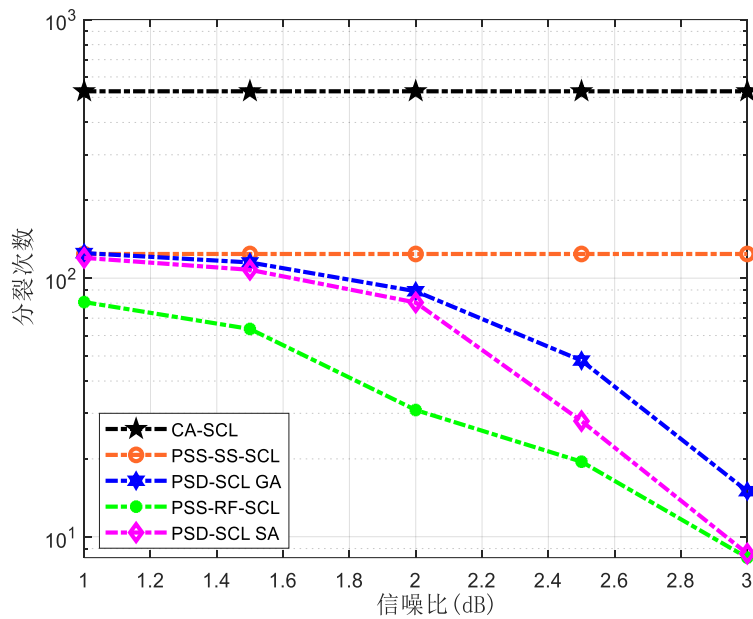


图 3-9 不同算法的平均分裂次数

Fig. 3-9 Average number of splitting operations for different algorithms

综上所述，本文所提算法在保持译码性能不低于 PSS-SS-SCL 译码算法性能的前提下，明显降低了译码复杂度。由于 PSS-RF-SCL 译码算法采用的基于 APM 值的剪枝策略，提前剪枝掉了明显错误路径使得错误路径没有参与后续译码，这带来了译码列表总数的明显下降。同时基于可靠性函数的不可靠比特选择策略为该算法带来了更精准的不可靠比特选择，使得分裂次数明显下降，而且译码性能十分稳定。

3.5 本章小结

本章先介绍了 PSS-SS-SCL 译码算法，对采用路径分裂选择的进步意义进行了介绍，并分析了进行路径分裂选择的核心——不可靠比特选择。然后分析了 PSS-

SS-SCL 译码算法的不足，并提出了改进的译码策略。其一是采用可靠性函数方法来识别需要进行路径分裂的比特。其二是基于 APM 值的路径剪枝策略。然后基于上述策略提出了一种基于可靠性函数的极化码 PSS-RF-SCL 译码算法。PSS-RF-SCL 译码算法采用了更符合译码实际情况的可靠性函数方法来识别需要进行路径分裂的比特。通过统计实验设定阈值 α ，只对可靠性函数值 $E(i)$ 小于 α 的信息比特进行路径分裂，明显降低了分裂次数。此外，通过统计实验分析得出了被淘汰的正确译码路径的 APM 值分布，得到了正确译码路径的 APM 值界限，并将其设为阈值 β 。运用路径 APM 值和阈值 β 来衡量路径的可靠性。对不可靠路径进行剪枝，降低了译码列表总数。最后通过仿真实验验证了本文提出的译码算法复杂度要低于现有的基于搜索集的路径分裂策略辅助 SCL 译码算法，并且能保持良好的译码性能。

第4章 候选路径选择策略辅助并行分裂简化译码算法的研究

4.1 本章引言

上一章中 PSS-RF-SCL 译码算法通过对不可靠比特和路径进行识别，只分裂和保留最有潜力的部分路径进行后续计算，以此减少了复杂度。这相当于为 SCL 译码算法提供了一个“做减法”的路径管理机制，这是在决策端的优化。但除了决策端，在译码计算过程中还存在一个计算端的矛盾点——即由串行递归计算带来的复杂度和时延。优化了决策端后，本文的第二个研究点将针对计算端的矛盾进行优化。在该方向上，Fast-SCL 译码算法利用极化码译码二叉树中存在的特殊节点，对这些节点进行一次性快速译码，从而跳过大量的递归计算。但 Fast-SCL 译码算法在获得叶子节点的码字后，依然需要进行串行路径分裂。因此本文将提出一种优化的并行路径分裂简化译码算法。本文首先将说明 Fast-SCL 译码算法的进步和不足，接着说明并行路径分裂遇到的问题，然后提出解决方案和优化后的并行路径分裂简化译码算法。最后进行仿真实验，证明该算法在不降低性能的前提下，在复杂度方面的优越性。

4.2 Fast-SCL 译码算法

最传统的对数似然比的计算方式是通过逐个比特串行计算。但陆续有学者发现在各种特殊节点上，可由中间节点直接得到叶子节点的对数似然比，并且在理论上证明其性能与串行递归计算是完全等价的。Fast-SCL 译码算法与 SCL 译码算法的不同之处就在于，其在特殊节点处利用中间节点直接估计得到了该节点处所有比特的值，省去了串行迭代计算的流程，降低了译码时延。Fast-SCL 译码算法主要采用了 4 种特殊结构的节点，如图 4-1 所示。

图 4-1 为码长 16 的译码二叉树。在该二叉树中，叶子节点根据比特类型进行了区分标注：其中黑色圆点代表信息比特，是承载有效信息的节点；白色圆点则代表冻结比特，其取值在编译码前已预先确定，不参与信息传输。图 4-1 中展示了四种具有特殊结构的节点类型，并标注了它们各自对应的对数似然比集合。这些特殊节点的显著特征在于，其叶子节点所对应的码字可直接依据其对数似然比集合进行估计，而无需沿二叉树结构逐层进行递归计算，从而在一定程度上简化了译码流程。然而需要指出的是，尽管上述特殊节点的引入能够减少递归调用的次数，但由于算法采用了多路径保留的译码策略，在每一个译码阶段仍需对所有保留路径进

行 PM 值的计算与更新。同时，在面临信息比特判决时，路径分裂操作依然不可避免，以保证潜在正确路径得以保留。因此，特殊节点的识别与利用主要作用于降低递归计算的频率，而并未改变多路径译码算法在路径度量管理与路径扩展方面的基本操作流程。

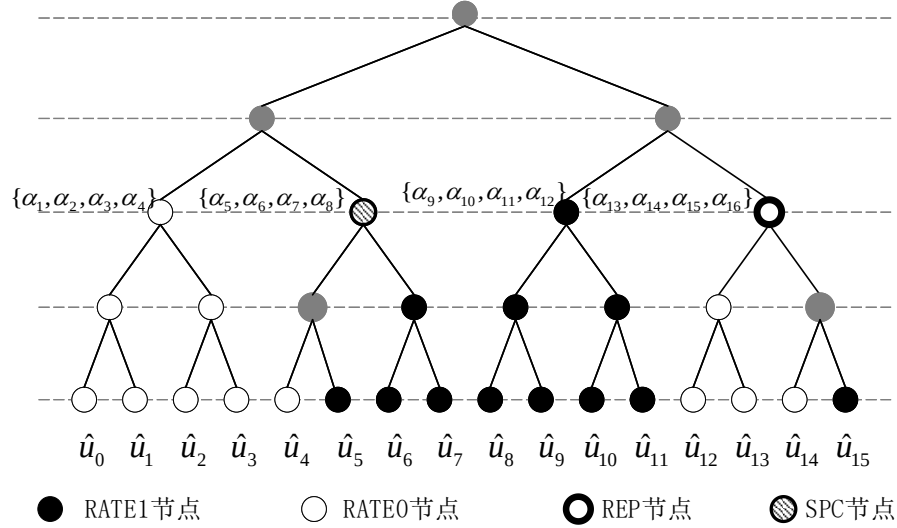


图 4-1 四种特殊节点的译码二叉树

Fig. 4-1 Decoding binary tree with four special nodes

以下是四种特殊节点计算 PM 值和路径分裂的详细过程：

(1) Rate-0 节点

Rate-0 节点全由冻结比特组成，不需要路径分裂，只需计算 PM 值，其计算方式如式(4-1)所示。

$$PM = PM_l + \sum_{i=1}^n \ln(1 + e^{-\alpha_i^l}) \quad (4-1)$$

(2) Rep 节点

在 Rep 节点上只有最后一个位置为信息比特，因此只需要进行一次路径分裂操作即可。并且分裂出的路径在 Rep 节点的叶子节点处要么为全零比特，要么为全一比特。两条路径的 PM 值计算如式(4-2)所示。

$$\begin{aligned} PM_l^0 &= PM_l + \sum_{i=1}^{N_v} \ln(1 + e^{-\alpha_i^l}) \\ PM_l^1 &= PM_l + \sum_{i=1}^{N_v} \ln(1 + e^{\alpha_i^l}) \end{aligned} \quad (4-2)$$

(3) Rate-1 节点

Rate-1 节点全为信息比特，如果全部进行路径分裂是不必要的，只需要按照节点对数似然比集合 $\{|\alpha_1|, |\alpha_2|, \dots, |\alpha_{N_v}|\}$ 的 LLR 大小进行排序，通过从小到大依次翻转 LLR 值对应的码字，完成译码路径的分裂。而翻转次数在文献[17]中已经确定为式(4-3)所示。

$$\min(L-1, N_v) \quad (4-3)$$

式中， L 为最大列表数， N_v 为该节点的叶子节点数。每次分裂后按式(4-4)更新 PM 值。

$$\begin{aligned} PM_l^0 &= PM_l \\ PM_l^1 &= PM_l + |\alpha_i'| \end{aligned} \quad (4-4)$$

(4) SPC 节点

SPC 节点中存在一个特点，即所有码字符符合偶校验。根据对数似然比集合判断出的码字如果符合偶校验，那么该码字是符合标准的，否则就需要将对数似然比集合 $\{|\alpha_1|, |\alpha_2|, \dots, |\alpha_{N_v}|\}$ 中 LLR 值最小的比特进行翻转得到一个基础码字，然后在该码字的基础上根据 $\{|\alpha_1|, |\alpha_2|, \dots, |\alpha_{N_v}|\}$ 的大小排序，进行总共 $\min(L-1, N_v)$ 次的翻转。其 PM 值更新如式(4-5)所示。

$$\begin{aligned} PM_l^0 &= PM_l \quad q=q \\ PM_l^1 &= PM_l + |\alpha_i'| + (1-2q)|\alpha_i'|, \quad q=\bar{q} \end{aligned} \quad (4-5)$$

式中，当根据对数似然比集合判断出的码字符符合偶校验时，令 $q=0$ ，否则 $q=1$ 。

根据上面的描述可知，Fast-SCL 译码算法虽然可以一次性获得节点中所有比特的对数似然比集合。但是进行路径分裂时，依旧需要串行的执行 $\min(L-1, N_v)$ 次比特翻转。这种下一次路径分裂等待上一次分裂完成的操作方式会增加时延。

4.3 一种候选路径选择策略辅助并行分裂简化译码算法

本节主要针对 Fast-SCL 译码算法 Rate-1 节点中的路径分裂方式进行更改，提出一种并行分裂简化译码算法。在进行并行路径分裂时，会带来候选路径数量过多的问题。过多的候选路径数量会造成排序、比较和存储的压力都较大，使译码复杂度升高。因此提出一种候选路径选择策略来减少候选路径的数量。

4.3.1 并行分裂方式

Fast-SCL 译码算法的串行路径分裂如图 4-2 所示。首先根据对数似然比集合直接判断出码字。然后根据 LLR 的排序进行 $\min(L-1, N_v)$ 次翻转，每次只翻转所

有路径的其中一个比特。图 4-2(a)中展示的是三个比特进行三次翻转，最大保留路径数为 4 时的情况，其中带线条部分代表被删除的路径。每条路径的翻转在上一次翻转完成后的情况下进行，并且 PM 值按照上一次累计后的结果继续进行计算。

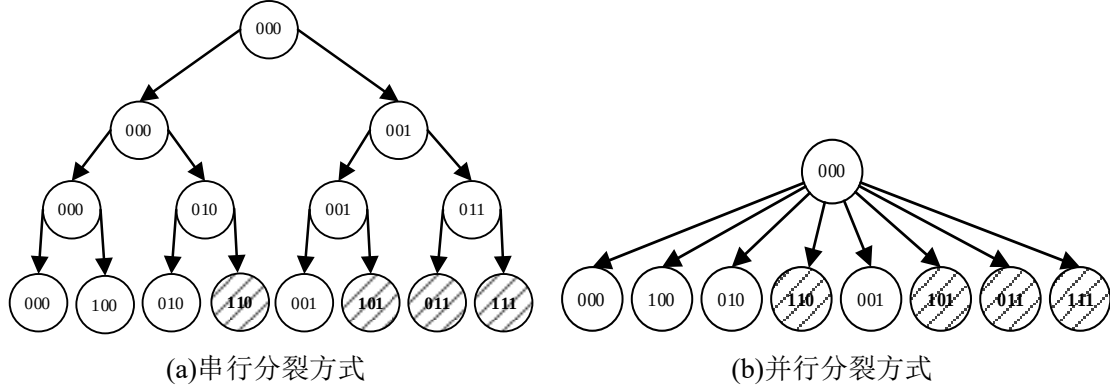


图 4-2 路径分裂方式

Fig. 4-2 Path splitting method

而并行分裂方式如图 4-2(b)所示，并行分裂一次性会得到分裂比特的所有情况，然后再从里面进行筛选。这样将图 4-2(a)中按三步进行的分裂操作变为只需执行一步操作，减少了步骤之间的等待时间，降低了译码时延。

但并行分裂也会产生一个明显的问题，其候选路径数量会明显上升。如果在串行路径分裂前路径数量已经达到最大列表数 L ，每次分裂 L 条路径会产生 $2L$ 条候选路径。然后会进行排序、删除操作，只保留前 L 条路径。所以总的候选路径访问数量如式(4-6)所示。

$$L_{\text{visit}} = L \cdot [\min(L, N_v) + 1] \quad (4-6)$$

而在并行分裂时，会得到分裂比特的所有情况，从 $\{0,0,0\dots,0\}$ 到 $\{1,1,1\dots,1\}$ 。产生的候选路径数量如式(4-7)所示。

$$L_{\text{visit}} = L \cdot 2^{\min(L, N_v)} \quad (4-7)$$

从式(4-7)和式(4-6)的对比可以看出串行分裂当中，候选路径访问数量与分裂次数是线性关系，而在并行分裂当中它们是指数级关系。因此在并行分裂当中的候选路径数量会明显大于串行分裂方式中的数量。并且 L 和 N_v 越大，差距也越大。

4.3.2 筛选候选路径的方式

将 Fast-SCL 译码算法改为采用并行分裂后，其候选路径数量会明显上升。多余的候选路径数量会造成排序、比较和存储的压力都较大，使译码复杂度升高。因此需要采用一种方式来对候选路径进行筛选，提前排除那些不可能进入最终列表

的候选路径，从而降低译码复杂度。Fast-SCL 进行路径分裂时会在每条路径上生成多条候选路径(通过在该路径上翻转多个不可靠比特位)。因为翻转比特会在路径度量值上施加惩罚，所以候选路径之间存在一定的偏序关系。例如，在路径分裂前排名第 L 的路径，它翻转任意比特产生的候选路径由于被施加了惩罚值排名必定低于 L 。因此分裂前排名第 L 的路径分裂产生的候选路径在排序中一定会被淘汰，所以该路径不需要进行路径分裂。同时其它路径分裂后产生候选路径的可能最好排名可通过式(4-8)进行粗略计算。

$$m_{l,j} = l + 2^x - 1 \quad (4-8)$$

式中 x 表示翻转比特的数量， l 是路径分裂前的排名。例如排名第 $L-1$ 的路径，如果翻转了一个比特，那么该路径的可能最好排名为 L 。但如果翻转了两个比特，该路径的可能最好排名为 $L+1$ ，因此如果需要翻转两个及以上比特数的候选路径就没必要产生。如图 4-3 所示是在最大列表数为 8，有 4 个翻转比特情况下的最好排名表。将翻转比特按对数似然比排序，并用二进制数表示，1 表示翻转，0 表示未翻转。如 0001 表示翻转对数似然比最小的比特。 j 表示该二进制数转换为十进制数的大小。

		翻转比特个数															
		0	1	1	2	1	2	2	3	1	2	2	3	2	3	3	4
路径 分裂 前 排名	8	8	9	9	11	9	11	11	15	9	11	11	15	11	15	15	23
	7	7	8	8	10	8	10	10	14	8	10	10	14	10	14	14	22
	6	6	7	7	9	7	9	9	13	7	9	9	13	9	13	13	21
	5	5	6	6	8	6	8	8	12	6	8	8	12	8	12	12	20
	4	4	5	5	7	5	7	7	11	5	7	7	11	7	11	11	19
	3	3	4	4	6	4	6	6	10	4	6	6	10	6	10	10	18
	2	2	3	3	5	3	5	5	9	3	5	5	9	5	9	9	17
	1	1	2	2	4	2	4	4	8	2	4	4	8	4	8	8	16
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
		j															

图 4-3 最好排名表

Fig. 4-3 Best ranking list

接着考虑更细致的情况。

(1) 以翻转一个比特作为例子，因为对数似然比排名的关系，以 0010 作为二进制数表示的翻转情况肯定会差于 0001 的翻转情况。因此在翻转相同比特数量的

情况下，其内部依然存在偏序关系。如图 4-4 给出的是总共五个比特，翻转其中两个比特的偏序关系。

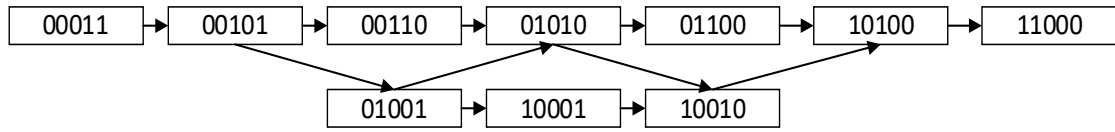


图 4-4 相同翻转数量下的偏序关系

Fig. 4-4 Partial order relationship under the same number of flips

从图 4-4 中可以看出，翻转相同比特数量，候选路径之间既存在从优到劣的关系，又存在一种彼此均衡的关系。

(2) 0011 的翻转情况无法判断是否比 1000 的翻转情况差，但 1001 的翻转情况肯定差于 1000。因此，当一种翻转情况可以完全包含另一种翻转情况并有多余翻转比特的情况下，其最好排名肯定更差。

在式(4-8)的基础上叠加一个判断情况如式(4-9)所示。

$$m_{i,j} = l + 2^x - 1 + z \quad (4-9)$$

式中 z 是对上述两种情况的一个判断，这进一步降低了某些候选路径的最小可能排名，从而允许排除更多候选路径。

如图 4-5 所示是更细致的最好排名表，其中最大列表数为 8，有 4 个翻转比特。

翻转比特个数

	0	1	1	2	1	2	2	3	1	2	2	3	2	3	3	4
8	8	9	10	11	11	12	13	15	12	13	14	16	15	17	18	23
7	7	8	9	10	10	11	12	14	11	12	13	15	14	16	17	22
6	6	7	8	9	9	10	11	13	10	11	12	14	13	15	16	21
5	5	6	7	8	8	9	10	12	9	10	11	13	12	14	15	20
4	4	5	6	7	7	8	9	11	8	9	10	12	11	13	14	19
3	3	4	5	6	6	7	8	10	7	8	9	11	10	12	13	18
2	2	3	4	5	5	6	7	9	6	7	8	10	9	11	12	17
1	1	2	3	4	4	5	6	8	5	6	7	9	8	10	11	16
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

j

路径分裂前排名

图 4-5 最好排名表

Fig. 4-5 Best ranking list

最后，考虑跟实际译码情况进行结合。在极化码译码当中，某个比特的对数似然比的绝对值越大，表明其对码字 $\hat{u}_i$ 的判决越有信心。在并行分裂当中，会同时翻转多个比特，因此需要考虑的不仅是对单个码字 $\hat{u}_i$ 判决的信心，而是需要同时考虑多个比特的组合情况。因此可以考虑当多个比特的对数似然比相加大于某个阈值时，该候选路径产生的必要性就较低。

关于该阈值的选取可以根据路径分裂前，输入路径的 PM 值进行计算。如果路径分裂前已经达到最大列表数 L ，那么 θ 表示可以接受的惩罚值的上限。 θ 值的计算如式(4-10)所示。

$$\theta = PM_L - PM_1 \quad (4-10)$$

如果产生某条候选路径增加的惩罚值大于 $PM_L - PM_1$ ，那么该路径的最好排名一定低于 L 。如果路径分裂前没有达到最大列表数 L ，就只通过式(4-9)来进行候选路径的筛选。

式(4-11)就是在分裂前，列表数已达到 L 时，在(4-9)偏序关系的基础上叠加当前比特的对数似然比之和的判断情况。

$$m_{l,j} = l + 2^x - 1 + z + a_l \quad (4-11)$$

式中 a_l 的值由式(4-12)计算得出。

$$a_l = \begin{cases} L & \sum_{i \in A} LLR_i > \theta \\ 0 & Else \end{cases} \quad (4-12)$$

式中 A 为翻转比特的集合。

根据式(4-11)生成的最好排名表并非固定不变，而是能够依据当前译码状态实时更新，从而动态反映各候选路径的优先级顺序。图 4-6 以随机示例的形式展示了该最好排名表在具体译码场景中的表现，其中最大列表数设置为 8，共计包含 5 个待翻转比特。从图 4-6 中可以看出，基于该最好排名表所生成的候选路径数量，相较于由式(4-7)直接计算得到的候选路径数量有了显著减少。并且与传统的串行分裂方式不同，传统方式依赖于通过 LLR 值计算各路径的 PM 值，进而依据 PM 值的大小关系进行排序以淘汰劣势路径，这一过程涉及大量数值计算与比较操作。而本文提出的方法则首先根据的是路径的偏序关系，路径的偏序关系是静态的，无需实时计算，仅需将相关数据预先存储，在需要时直接调用即可，从而大幅降低了在线计算开销。在当前路径列表规模已达到最大列表数 L 的情况下，算法将叠加对翻转比特的对数似然比之和的判断，该判断过程仅需将计算结果与阈值 θ 进行比较。

相较于传统方法中需要通过路径间多次排序比较才能确定保留路径的方式，这种比较的机制简化了判断流程，有效降低了译码复杂度。

翻转比特个数

	0	1	1	2	1	2	2	3	1	2	2	3	2	3	3	4	1	2	2	3	2	3	3	4	2
8	8	9	10	11	11	12	13	14	12	13	14	15	15	16	17	18	13	14	15	16	16	17	18	19	17
7	7	8	9	10	10	11	12	13	11	12	13	14	14	15	16	17	12	13	14	15	15	16	17	18	16
6	6	7	8	9	9	10	11	12	10	11	12	13	13	14	15	16	11	12	13	14	14	15	16	17	15
5	5	6	7	8	8	9	10	11	9	10	11	12	12	13	14	15	10	11	12	13	13	14	15	16	14
4	4	5	6	7	7	16	17	18	16	17	18	19	19	20	21	22	17	18	19	20	20	21	22	23	21
3	3	4	5	6	6	15	16	17	15	16	17	18	18	19	20	21	16	17	18	19	19	20	21	22	20
2	2	3	4	5	5	14	15	16	14	15	16	17	17	18	19	20	15	16	17	18	18	19	20	21	19
1	1	2	3	12	12	13	14	15	13	14	15	16	16	17	18	19	14	15	16	17	17	18	19	20	18
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

j

路径分裂前排名

图 4-6 最好排名表

Fig. 4-6 Best ranking list

4.3.3 PS-PSSCL 译码算法流程

在 Fast-SCL 译码算法的基础上，为减少译码复杂度，在 Rate-1 节点上实行路径的并行分裂。并且为了减少候选路径数量，采用了路径偏序关系叠加翻转比特的对数似然比之和的策略进行候选路径筛选。其译码具体过程如算法 4-1 所示。

算法 4-1 PS-PSSCL 译码算法

START

Input: 码长 N ，接收端序列，静态最好排名表，节点矩阵 **Node_Matrix**

Output: 译码估计序列

1: FOR i_node : **Node_Matrix**

2: IF $i_node == \text{Rate-0}$

3: $\hat{\mathbf{u}} = \mathbf{0}$

4: 更新 PM 值 $PM = PM_l + \sum_{i=1}^n \ln(1 + e^{-\alpha_i^l})$

5: ELSE IF $i_node == \text{Rep}$

6: 分裂出两条路径 $\hat{\mathbf{u}} = \mathbf{0}$ ， $\hat{\mathbf{u}} = \mathbf{1}$

7: IF $\hat{\mathbf{u}} = \mathbf{0}$

8: $PM_l^0 = PM_l + \sum_{i=1}^{N_{\hat{\mathbf{u}}}} \ln(1 + e^{-\alpha_i^l})$

9: ELSE IF $\hat{\mathbf{u}} = \mathbf{1}$

```

10:  $PM_l^1 = PM_l + \sum_{i=1}^{N_v} \ln(1 + e^{\alpha_i^l})$ 
11: END IF
12: ELSE IF  $i\_node == Rate-1$ 
13: 硬判决得到码字  $\hat{\mathbf{u}}$ 
14: 设置翻转次数  $\min(L-1, N_v)$ 
15: 根据静态最好排名表开始生成候选路径
16: 生成候选路径过程中根据翻转比特的对数似然比之和来筛选候选路径, 如果
 $\sum_{i \in A} LLR_i > \theta$ , 该候选路径不会被生成
17: IF  $\hat{\mathbf{u}} = \mathbf{0}$ 
18:  $PM_l^0 = PM_l$ 
19: ELSE IF  $\hat{\mathbf{u}} = \mathbf{1}$ 
20:  $PM_l^1 = PM_l + |\alpha_i^l|$ 
21: END IF
22: 候选路径通过排序、删除操作得到最后的保留路径
23: ELSE IF  $i\_node == SPC$ 
24: 硬判决得到码字  $\hat{\mathbf{u}}$ 
25: 设置翻转次数  $\min(L-1, N_v)$ 
26: 依次分裂路径, 通过排序、删除操作得到最后的保留路径
27: END IF
28: END FOR

```

PS-PSSCL 译码算法相较于 Fast-SCL 译码算法的优化主要体现在 Rate-1 节点的译码过程中。该算法首先以节点接收到的对数似然比序列值的硬判决结果作为最大似然译码的初始估计, 在此基础上确定待翻转的比特。随后, 根据预设的最大列表数 L 与节点中包含的信息比特数量 N_v , 计算得到翻转次数 $\min(L-1, N_v)$, 该值决定了在 Rate-1 节点内参与路径分裂的比特数目。接下来, 算法依据预先通过式(4-9)构建的路径偏序关系开始候选路径的生成。该路径偏序关系是在译码开始前就已计算完成的静态结构, 无需在译码过程中重复计算, 仅需根据当前设定的列表数 L 及实际翻转比特数量, 从中直接读取最好排名高于或等于 L 的路径及其对应的翻转位置信息。同时在候选路径生成的过程中对翻转比特的对数似然比之和进行判断, 对数似然比之和大于阈值 θ 的候选路径不会生成, 从而高效地确定待扩展的候选路径集合。最后, 算法对所有生成的候选路径进行统一的排序与删除操作, 从中筛选出保留路径进入后续译码阶段。

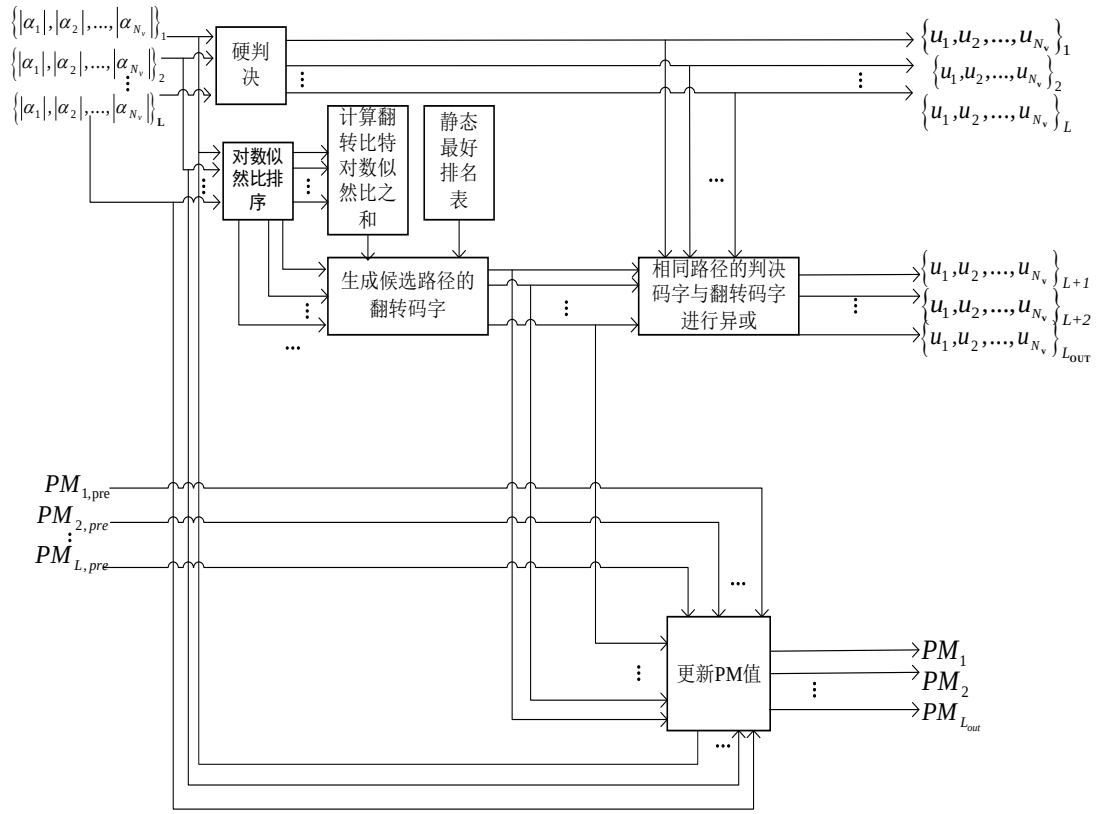


图 4-7 并行分裂译码结构

Fig. 4-7 Parallel split decoding structure

图 4-7 为并行分裂译码结构，输入为各条译码路径的对数似然比集合 $\{|\alpha_1|, |\alpha_2|, \dots, |\alpha_{N_v}|\}$ 以及路径度量值 PM_{pre} 。输出为候选路径的码字，总共的候选路径数量为 L_{out} 。该候选路径数量 L_{out} 是根据静态最好排名表以及翻转比特的对数似然比之和进行判断得到的。“生成候选路径的翻转码字”模块的输出为一串 0、1 序列 $\{\sigma_1, \sigma_2, \dots, \sigma_{N_v}\}$ ，1 代表进行翻转，0 代表不翻转。该序列的索引值与硬判决码字的索引值一一对应。在后面的“异或”模块中，硬判决码字 $\{u_1, u_2, \dots, u_{N_v}\}$ 将与对应的翻转码字进行按位异或得到候选路径的码字。“更新 PM 值”模块的输入为并行分裂前的路径度量值 $PM_{i,pre}$ 、候选路径的翻转码字 $\{\sigma_1, \sigma_2, \dots, \sigma_{N_v}\}$ 、对数似然比集合 $\{|\alpha_1|, |\alpha_2|, \dots, |\alpha_{N_v}|\}$ 。其输出为经过翻转的候选路径的 PM 值，其计算方式如式(4-13)所示。

$$PM_i = PM_{i,pre} + \sum_{j=1}^{N_v} \sigma_j \cdot |\alpha_j| \quad (4-13)$$

由硬判决产生的候选路径的 PM 值由式(4-14)计算得到。

$$PM_i = PM_{i,pre} \quad (4-14)$$

通过式(4-13)和式(4-14)计算得到 PM 值后就可以进行后续的排序、删除操作。

4.3.4 仿真分析

为验证本章所提出的 PS-PSSCL 译码算法的优越性,与文献[24]的 PO-Fast-SCL 译码算法、EXPOS-Fast-SCL 译码算法、文献[18]的 Fast-SCL 译码算法进行译码复杂度和误块率性能的比较。其中 PO-Fast-SCL 译码算法和 EXPOS-Fast-SCL 译码算法都是在 Rate-1 节点上执行并行路径分裂的译码算法,只是它们筛选候选路径的方式不同。Fast-SCL 译码算法则是在 Rate-1 节点上执行串行路径分裂的译码算法,可以当作 BLER 性能的标杆。

为保证比较的公平性,所有算法设置相同的码长 N , 码率 R , 最大列表数 L , 添加相同长度 CRC 码, 并且每个算法在 Rate-1 节点中都进行了 $\min(L-1, N_v)$ 个比特的翻转。各个算法均采用 $SNR = 2.5dB$ 的 GA 构造方法和 BPSK 调制, 信道选用 AWGN 信道。

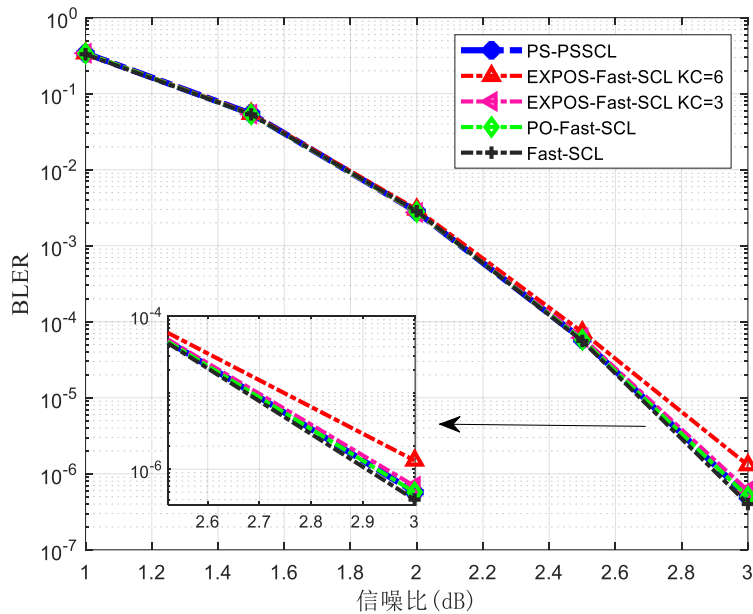


图 4-8 $N=1024, R=1/2, L=8$ 时各算法误块率仿真图

Fig. 4-8 Simulation graph of block error rate for each algorithm at $N=1024, R=1/2, L=8$

如图 4-8 所示, 在码长 $N=1024$, 码率 $R=1/2$, 列表数 $L=8$ 的情况下, 比较各个算法的误块率性能, 其中 EXPOS-Fast-SCL 译码算法的梯度 k_c 分别设为 3 和 6。EXPOS-Fast-SCL 译码算法的梯度 k_c 代表的是该算法削减候选路径的程度, k_c 值越大, 候选路径的数量越少, 复杂度越低, 但性能损失越明显。为合理选取 k_c 值, 本文分别从复杂度层面和性能层面出发, 使不同 k_c 值下的 EXPOS-Fast-SCL 译码算

法可以贴近所提算法的相应指标。从图 4-8 中可以分析得出所提出的 PS-PSSCL 译码算法与 PO-Fast-SCL 译码算法相较于 Fast-SCL 译码算法几乎没有性能损失, $k_c=3$ 时的 EXPOS-Fast-SCL 译码算法有略微的性能损失。而 $k_c=6$ 时的 EXPOS-Fast-SCL 译码算法性能损失比较明显, 在 $BLER=10^{-5}$ 处有 0.08dB 的性能损失。

为了验证所提算法在不同列表数下, 都能达到几乎与 Fast-SCL 译码算法一样的 BLER 性能。图 4-9 展示了与图 4-8 相比, 不同列表数下的 BLER 性能仿真图。图 4-9 的列表数 L 设置为 4, 其余设置均与图 4-8 相同。在图 4-9 中, 所提出的 PS-PSSCL 译码算法相较于 Fast-SCL 译码算法几乎没有性能损失。并且只有 $k_c=4$ 时的 EXPOS-Fast-SCL 译码算法有明显的性能损失, 在 $BLER=10^{-4}$ 处有 0.1dB 的性能损失。虽然 PO-Fast-SCL 译码算法和 $k_c=3$ 时的译码算法与本文所提出的算法误块率性能接近。但是它们在 Rate-1 节点中产生的候选路径数量要多于本文所提出的算法, 这会在后续的复杂度比较中进行说明。

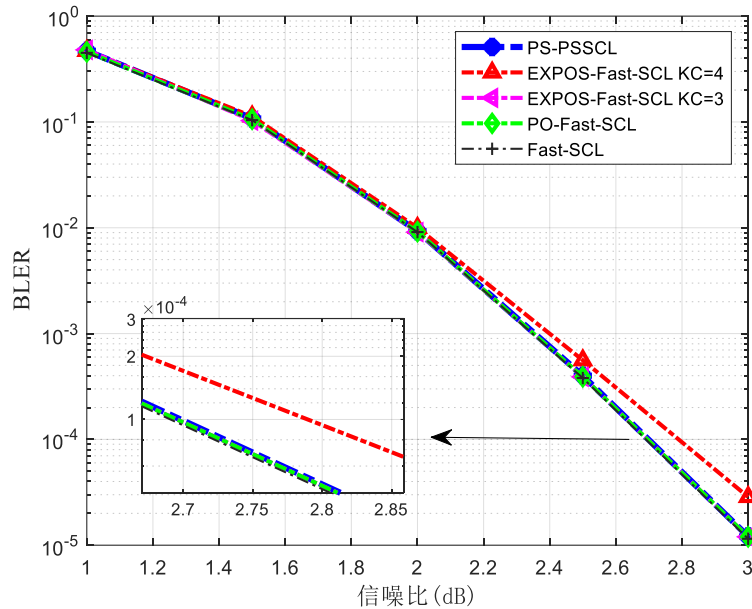
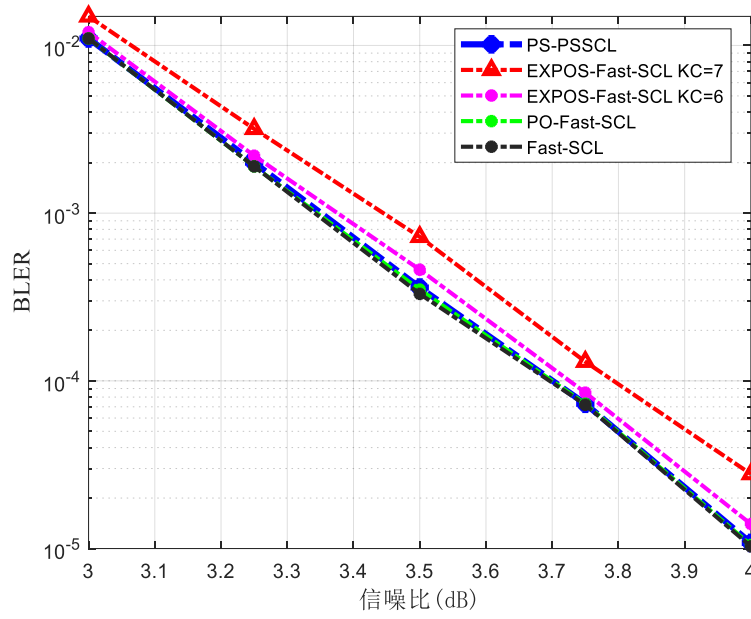


图 4-9 $N=1024, R=1/2, L=4$ 时各算法误块率仿真图

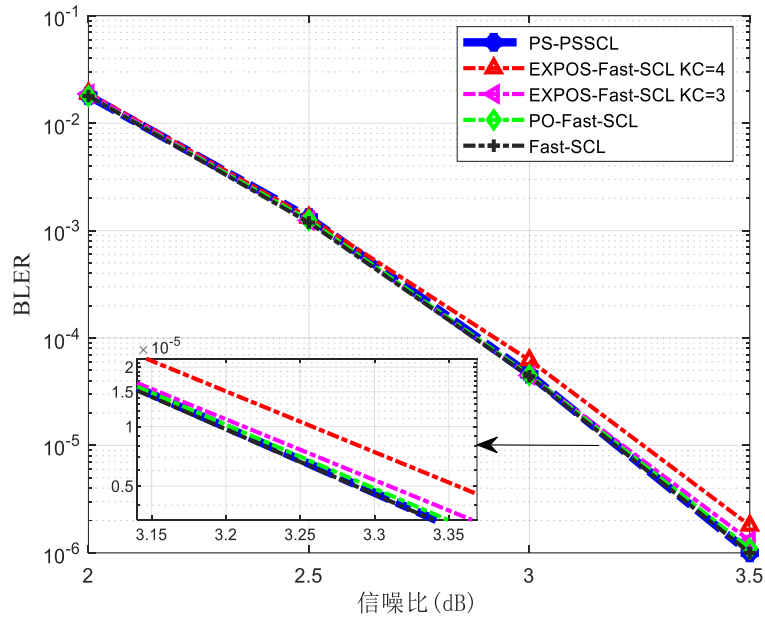
Fig. 4-9 Simulation graph of block error rate for each algorithm at $N=1024, R=1/2, L=4$

极化码的码长和码率对极化码的性能也有明显的影响。为验证本文所提算法即使在不同的码长和码率下都能保持性能几乎没有损失, 因此采用了不同的码长和码率进行了仿真对照实验。

如图 4-10 所示, 在码长 $N=1024$, 码率 $R=3/4$, 列表 $L=8$ 的情况下, 比较各个算法的误块率性能。

图 4-10 $N=1024, R=3/4, L=8$ 时各算法误块率仿真图Fig. 4-10 Simulation graph of block error rate for each algorithm at $N=1024, R=3/4, L=8$

从图 4-10 中可看出，本文提出的算法在高码率下性能与 Fast-SCL 译码算法几乎一致。 $k_c = 6$ 时的 EXPOS-Fast-SCL 译码算法有略微的性能损失。而 $k_c = 7$ 时的 EXPOS-Fast-SCL 译码算法性能损失较大，在 $BLER = 10^{-4}$ 时，有 0.09dB 的性能损失。

图 4-11 $N=512, R=1/2, L=8$ 时各算法误块率仿真图Fig. 4-11 Simulation graph of block error rate for each algorithm at $N=512, R=1/2, L=8$

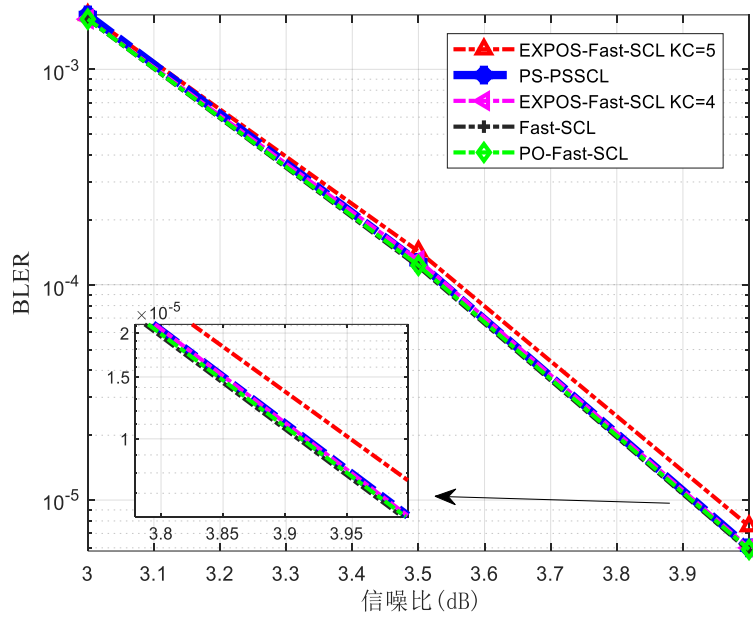
图 4-12 $N=256, R=1/2, L=8$ 时各算法误块率仿真图Fig. 4-12 Simulation graph of block error rate for each algorithm at $N=256, R=1/2, L=8$

图 4-11 和图 4-12 分别是在码长为 512 和 256 情况下的误块率仿真图。从这两张图中可以看出无论是在中等码长 512 还是在短码长 256 的情况下，所提出的算法相较于 Fast-SCL 译码算法几乎都没有性能损失。在码长为 512 时， $k_c = 4$ 的 EXPOS-Fast-SCL 译码算法在 $BLER = 10^{-5}$ 处，有 0.05dB 的性能损失。在码长为 256 时， $k_c = 5$ 的 EXPOS-Fast-SCL 译码算法在 $BLER = 10^{-5}$ 处，有 0.04dB 的性能损失。

从误块率性能的分析中得出，所提出的算法在各种码长、码率和列表数的条件下均具备良好的性能，并不会出现明显的性能损失。为进一步验证所提算法的优越性，接下来将进行复杂度的分析。

在码长 $N=1024$ 的条件下，表 4-1 对比了不同译码算法的时间步长。从表 4-1 中可以看出，在 Rate-1 节点中采用并行分裂策略的译码算法，其时间步长明显低于未引入并行分裂机制的 Fast-SCL 译码算法。具体而言，在码率为 1/2、列表数为 8 的情况下，本文提出的 PS-PSSCL 译码算法相较于 Fast-SCL 译码算法，时间步长降低了 12%。当系统采用高码率和大列表数配置时，这一性能优势表现得更加明显。这是因为在高码率条件下，Rate-1 节点所包含的信息比特数量较多，为并行处理提供了更大的优化空间。在大列表数的设置下，需要进行路径分裂的比特数量增加，并行分裂策略能够更充分地发挥其并行处理优势。特别地，当码率提升至 3/4，列表数增加到 32 时，PS-PSSCL 译码算法相比 Fast-SCL 译码算法的时间步长下降幅度达到 41%。这里的时间步长定义为译码器完成整个译码过程所需的基本操作单元数量，它是衡量译码算法内在延迟特性的关键指标。在 SCL 译码算法框架下，

无论是任意节点的对数似然比计算，还是信息比特节点的路径分裂操作，均被计算为一个基本操作单元。因此，时间步长的缩短意味着在算法层面上能够有效降低译码延迟^[18]。

表 4-1 码长 $N=1024$ 时各算法的时间步长

Table4-1 The time step of each algorithm when the code length $N=1024$

码率	L	PS-PSSCL 的时间步长	CA-SCL 的时间步长	Fast-SCL 的时间步长	EXPOS-Fast-SCL 的时间步长	PO-Fast-SCL 的时间步长
1/4	4	298	2302	316	298	298
1/4	8	334	2302	372	334	334
1/4	16	366	2302	415	366	366
1/4	32	398	2302	448	398	398
1/2	4	376	2558	400	376	376
1/2	8	436	2558	492	436	436
1/2	16	492	2558	583	492	492
1/2	32	540	2558	650	540	540
3/4	4	293	2814	333	293	293
3/4	8	333	2814	433	333	333
3/4	16	373	2814	557	373	373
3/4	32	405	2814	689	405	405

PO-Fast-SCL、EXPOS-Fast-SCL 算法与本文所提出的算法均在 Rate-1 节点中引入了并行分裂机制，显著提升了译码流程的并行度，因此它们与本文所提算法在关键的时间步长指标上表现一致。但这种并行分裂带来的时间步长减少是有代价的。具体而言，在 Rate-1 节点的并行处理过程中，多个比特位在同一时间步长内同时进行路径拓展，导致候选路径数量呈倍数级瞬时激增。这些大量衍生的多余路径随即对译码器造成了明显的压力。一方面，排序网络需对数量急剧膨胀的路径进行排序、比较和删除操作，另一方面路径存储器需要同时容纳并管理这些激增的候选路径。因此，Rate-1 节点中并行分裂所产生的候选路径数量，成为了衡量该类算法复杂度的重要指标。

如表 4-2 所示，是各个算法在 Rate-1 节点中产生的候选路数量的对比。PS-PSSCL 译码算法生成的候选路径数量跟实际的传输情况有关，在不同信噪比下其候选路径数量不同。因此取其在一定信噪比范围内的平均值和最大值。该信噪比范围与图 4-8~图 4-12 中所取的信噪比范围一致，因此可以方便的比较各个算法的性能和复杂度情况。EXPOS-Fast-SCL 算法的 k_c 值为削减候选路径的梯度值， k_c 越大则生成的候选路径数量越少，但性能损失也会更多。从表 4-1 中，分析得出无论是 PS-PSSCL 译码算法的候选路径数量平均值还是最大值都明显低于 PO-Fast-SCL 译码算法。从 BLER 性能分析中得出，PO-Fast-SCL 译码算法的性能虽然与所提出的

PS-PSSCL 译码算法性能几乎一致，但其复杂度却明显更高。EXPOS-Fast-SCL 算法在取表 4-2 中较高的 k_c 值时，其候选路径数量虽然低于 PS-PSSCL 译码算法的最大值，但其性能损失十分明显。并且在取表中较高的 k_c 值时其候选路径数量依然大于 PS-PSSCL 译码算法的平均值。EXPOS-Fast-SCL 算法在取表中较低的 k_c 值时，其性能损失较小，但候选路径数量却又高于 PS-PSSCL 译码算法的最大值和平均值，复杂度更高。从性能和候选路径数量两方面共同分析，EXPOS-Fast-SCL 算法想要达到与所提出的 PS-PSSCL 译码算法一样几乎没有性能损失的情况，其产生的候选路径数量要更多，复杂度更高。同时，EXPOS-Fast-SCL 算法的 k_c 值需要在传输之前通过迭代逼近实验来选取。也就是从最大的梯度值 L 开始迭代实验，不断减少直到获得能够接受的性能损失为止。因此 EXPOS-Fast-SCL 算法对不同传输环境的适应性并不好，而 PS-PSSCL 译码算法在不同传输环境中更加灵活。

表 4-2 各算法在 Rate-1 节点中的候选路径数量

Table4-2 The number of candidate paths for each algorithm in Rate-1 nodes

码长	码率	L	PS-PSSCL 的候选路径数	PO-Fast-SCL 的候选路径数	EXPOS-Fast-SCL 的候选路径数
1024	1/2	8	平均值(1dB~3dB):21.6 最大值(1dB~3dB):25.8	52	(Kc=3)28 (Kc=6)24.8
1024	1/2	4	平均值(1dB~3dB):7.3 最大值(1dB~3dB):7.8	10.8	(Kc=3)8.8 (Kc=4)7.8
1024	3/4	8	平均值(3dB~4dB):21.7 最大值(3dB~4dB):22.9	53	(Kc=6)24.2 (Kc=7)23.2
512	1/2	8	平均值(2dB~3.5dB):22.3 最大值(2dB~3.5dB):24	44.4	(Kc=3)24.5 (Kc=4)22.5
256	1/2	8	平均值(2dB~4dB):23.9 最大值(2dB~4dB):26.5	54.5	(Kc=4)27 (Kc=5)26

综上所述，本文所提出的 PS-PSSCL 译码算法能够在几乎不引入性能损失的前提下，以更低的候选路径数量高效地实现 Rate-1 节点中的并行路径分裂，从而明显地减少了译码过程中的时间步长。该算法通过对节点内路径分裂过程进行优化，在保证译码性能的同时，有效降低了路径比较与选择所带来的计算开销。相较于 EXPOS-Fast-SCL 算法和 PO-Fast-SCL 算法，PS-PSSCL 算法在几乎没有译码性能损失的情况下，所需候选路径数量更少，整体计算复杂度降低。实验结果表明，所提算法适用于对时延和资源受限的通信系统。

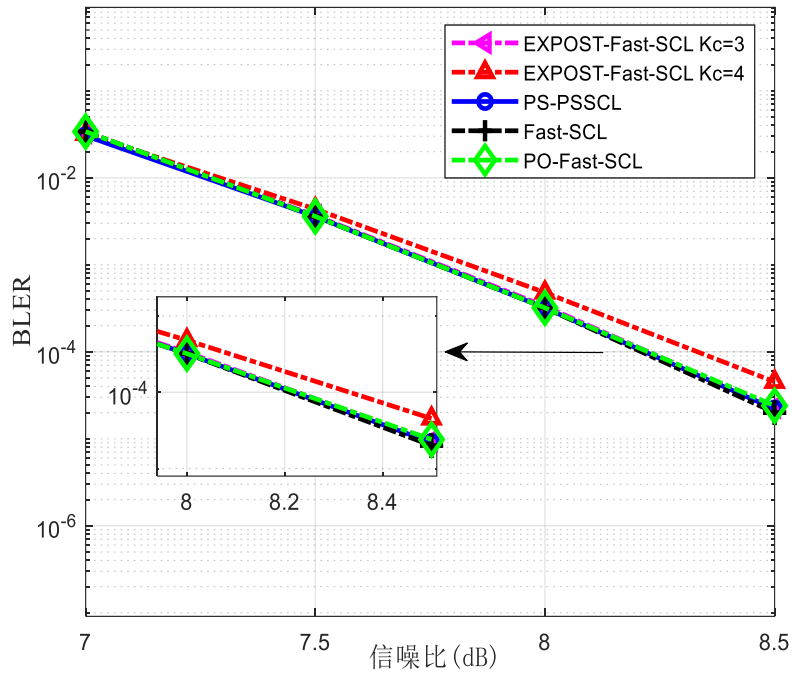


图 4-13 $N=1024, R=1/2$ 时不同译码算法在瑞利衰落信道下的误块率仿真图

Fig. 4-13 Simulation diagram of block error rate for different decoding algorithms with $N=1024$ and $R=1/2$ under Rayleigh fading channel

为了全面评估本文所提出的 PS-PSSCL 译码算法在不同信道环境中的泛化能力, 本文进一步对比了不同算法在瑞利衰落信道下的性能表现, 相应的对比结果如图 4-13 所示。结合图 4-8 所展示的高斯白噪声信道下的实验结果可以看出, 本文所提算法在两种典型信道条件下均表现出较好的稳定性。具体而言, 无论是在高斯白噪声信道还是瑞利衰落信道中, PS-PSSCL 译码算法的误块率性能均能紧密跟随 Fast-SCL 译码算法的基准曲线, 几乎没有性能损失, 同时保持了较低的复杂度, 其平均候选路径数量稳定在 7.6。相同的情况下, EXPOS-Fast-SCL 译码算法在 $k_c = 3$ 时虽然也能在性能上逼近 Fast-SCL 译码算法, 无明显损失, 但其平均候选路径数量为 8.8, 高于本文所提算法, 反映出较高的译码复杂度。当 k_c 值提高至 4 时, 该算法的平均候选路径数量虽然下降为 7.8, 复杂度有所降低, 但却表现出了明显的性能退化。在 $BLER = 10^{-4}$ 处, $k_c = 4$ 的 EXPOS-Fast-SCL 译码算法相较于 Fast-SCL 译码算法存在约 0.12dB 的性能损失。PO-Fast-SCL 译码算法的平均候选路径数量为 10.8, 在所有参与对比的并行分裂算法中居于首位, 这意味着其在实际应用中可能引入更高的资源占用。因此, 分析得出本文提出的 PS-PSSCL 译码算法在不同信道下也能拥有较好的性能和复杂度表现。

4.4 本章小结

本章针对 Fast-SCL 译码算法在 Rate-1 节点中的不足和在并行路径分裂中遇到的问题，提出了一种优化的并行路径分裂简化译码算法。该算法在 Rate-1 节点中使用并行路径分裂来减少时间步长，在翻转相同的 $\min(L-1, N_v)$ 个比特的情况下，将该步骤的时间步长降低为 1。同时使用路径偏序关系叠加翻转比特的对数似然比之和的策略进行候选路径筛选，达到减少候选路径数量的目的。仿真结果表明：本文提出的 PS-PSSCL 译码算法相较于其它在 Rate-1 节点中实行并行路径分裂的译码算法，在几乎没有性能损失的情况下能够以更低的候选路径数量完成 Rate-1 节点中的并行路径分裂，有效地降低译码复杂度。

第5章 总结与展望

5.1 主要工作及结论

极化码是被理论证明达到信道容量的信道编码方案，自提出以来凭借其特有的信道极化特性，在编码领域得到了极高的理论效应。由于其构造的特点，它在无限码长条件下的性能可以接近香农极限。但是在实际的通信系统中由于硬件资源与延迟的限制，码长是有限的。因此极化码在标准的 SC 译码器下性能明显落后于已经被实际系统广泛采用的 LDPC 码和 Turbo 码，因此制约了其早期的工程应用。在这种性能缺陷下，有学者提出了 CA-SCL 译码算法，在译码过程中保留多条候选路径，大幅提升了有限码长条件下极化码的纠错能力，并引入 CRC 码进行校验，让其性能达到甚至超越传统编码方案。然而，列表译码机制在估计每一个比特的时候需要进行路径的分裂和择优，在实际应用过程中会引入更高的计算复杂度和更长的译码时间，因此在 CA-SCL 译码算法作为基线算法的情况下，降低其复杂度有着重要的研究意义。但目前低复杂度的 SCL 译码算法仍然具有以下问题：1、PSS-SS-SCL 译码算法通过搜索集来进行不可靠比特的识别，并没有分析当前译码比特的实际情况，造成搜索集包含的不可靠比特数量较大，译码复杂度较高。并且 PSS-SS-SCL 译码算法只考虑了译码比特的可靠性，没有考虑到不同路径之间也存在可靠性的差别，这使得许多不必要的译码路径继续占用译码资源，使得译码复杂度升高。2、Fast-SCL 译码算法虽然可以一次性获得节点中所有比特的对数似然比集合。但在 Rate-1 节点中进行路径分裂时，依旧需要串行的执行 $\min(L-1, N_v)$ 次比特翻转。这种下一次路径分裂等待上一次分裂完成的操作方式会增加时延。并且现存的在 Rate-1 节点中执行并行分裂的译码算法在性能没有损失的情况下，产生的候选路径数量偏高。针对这两个问题，本文提出了 PSS-RF-SCL 译码算法和 PS-PSSCL 译码算法作为相应的解决方案。本文所做的工作可以总结如下：

1、针对 PSS-SS-SCL 译码算法存在的问题，提出了一种 PSS-RF-SCL 译码算法。在译码过程中，所提出的 PSS-RF-SCL 译码算法会在每一个信息比特进行路径分裂之前，先计算当前保留译码路径的路径度量值，并基于这些路径度量值计算该比特位置的可靠性函数值。通过将该比特的可靠性函数值与设定的阈值 α 进行比较，若其值低于 α ，则判定该比特为不可靠比特，需要进行路径分裂操作。反之，则说明该比特具有较高的可靠性，无需进行额外分裂。该机制能够有效避免在可靠性较高的比特位置上执行不必要的路径分裂，从而显著减少冗余的分裂次数，提升译码过程的效率。同时，为控制所保持的译码路径数量，算法进一步引入了一种可

以更好判断某条路径是否已经明显出错的评估指标——APM 值，并将其与阈值 β 进行比较。当某条路径的 APM 值高于 β 时，意味着该路径偏离正确译码路径的可能性较大，因此被判定为不可靠路径，并进行剪枝处理，即从候选列表中移除该路径。该策略在尽可能保留正确译码路径的同时，有效剔除了低可靠性路径，从而降低了译码列表的总数，控制搜索空间。仿真实验结果表明，与传统的基于搜索集的路径分裂策略辅助 SCL 译码算法相比，本文提出的 PSS-RF-SCL 译码算法在保持相同译码性能的情况下，可以有效的降低译码复杂度。该算法可以动态地识别分裂比特并剪枝不可靠路径，在复杂度与性能之间保持更好的平衡。

2、针对 Fast-SCL 译码算法在处理 Rate-1 节点时所暴露出的局限性和在并行路径分裂中遇到的问题，提出了一种候选路径选择策略辅助并行路径分裂简化译码算法。该算法的核心思想在于，在保证译码并行度的前提下，通过对 Rate-1 节点中的路径分裂机制进行优化设计来降低产生的候选路径数量。在 Rate-1 节点的译码过程中，该算法采用了并行路径分裂策略，能够在单个时间步长内同时完成对多个比特的路径拓展，从而将与翻转比特数量相关的串行操作压缩至单一时间步长内完成，在算法层面降低了译码延迟。为了应对并行分裂所引发的候选路径数量激增问题，算法进一步引入了一种融合路径偏序关系与翻转比特对数似然比之和的筛选机制。该机制利用偏序关系对候选路径进行结构性剪枝，同时结合对数似然比之和作为度量准则，从而在不影响纠错性能的前提下，有效缩减候选路径的规模。仿真结果表明，相较于其它同样在 Rate-1 节点中采用并行路径分裂的译码算法，所提出的 PS-PSSCL 算法能够在保持几乎无损的误块率性能的同时，以更低的候选路径数量完成并行分裂过程，从而在源头上缓解了排序、删除操作与存储单元的压力，实现了译码复杂度的有效降低。

5.2 研究展望

本文针对极化码低复杂度 SCL 译码算法进行了详细研究。但由于个人的时间和水平有限，面对不断演进的通信技术，本课题对极化码译码算法的研究还存在着一些不足之处需要完善：

1、本文提出的 PSS-RF-SCL 译码算法针对不同码长和码率的传输情况需要获取先验数据，即阈值 α 和阈值 β 的值，这限制了算法对不同传输情况的适应性，导致算法的灵活性降低。后续可以通过提出更好的比特和路径可靠性的衡量方式来调整路径分裂与剪枝策略，从而进一步提升算法在不同传输情况下的灵活性。

2、本文提出的 PS-PSSCL 译码算法主要针对关键的 Rate-1 节点译码方式进行了优化,后续对于其它特殊节点可以提出更优良的译码方式,从而进一步提升算法在不同传输情况下的灵活性。

3、本文提出的两种译码算法都是理论研究,通过软件仿真验证译码算法的性能优势,后续可以从实际应用层面出发,对所提算法进行硬件实现,评估其在实际芯片中的吞吐率、功耗与面积开销,以期在保持理论性能优势的同时,满足工程应用对实时处理与低功耗的严苛要求。

参考文献

- [1] ARIKAN E. Channel polarization: a method for constructing capacity-achieving codes for symmetric binary-input memoryless channels[J]. IEEE Transactions on Information Theory, 2009, 55(7): 3051–3073.
- [2] NIU K, CHEN K. CRC-aided decoding of polar codes[J]. IEEE Communications Letters, 2012, 16(10): 1668-1671.
- [3] MORI R, TANAKA T. Performance of polar codes with the construction using density evolution[J]. IEEE Communications Letters, 2009, 13(7): 519-521.
- [4] Tal I, VARDY A. How to construct polar codes[J]. IEEE Transactions on Information Theory, 2013, 59(10): 6562–6582.
- [5] OLIVEIRA R M, DE LAMARE R C. Polar codes based on piecewise gaussian approximation: design and analysis[J]. IEEE Access, 2022, 10: 73571-73582.
- [6] HE G, BELFIORE J C, LAND I, et al. Beta-expansion: a theoretical framework for fast and recursive construction of polar codes[C]. GLOBECOM 2017-2017 IEEE Global Communications Conference, Singapore, 2017: 1-6.
- [7] SARKIS G, GIARD P, VARDY A, et al. Fast list decoders for polar codes[J]. IEEE Journal on Selected Areas in Communications, 2016, 34(2): 318-328.
- [8] ZHAO Y, YIN Z, YANG Z, et al. Generalized constituent nodes for simplified successive-cancellation decoding of multi-kernel polar codes[J]. IEEE Communications Letters, 2023, 27(5): 1272-1276.
- [9] ZHOU Y, ZHENG Y, WANG Z. Fast successive-cancellation decoding of 5G parity-check polar codes[J]. IEEE Communications Letters, 2023, 27(1): 37-40.
- [10] 袁建国, 方小倩, 梁栩珩, 等. 一种基于均匀量化的快速简化极化码 SC 译码算法[J]. 半导体光电, 2021, 42(1): 127-131.
- [11] TAL I, VARDY A. List decoding of polar codes[J]. IEEE Transactions on Information Theory, 2015, 61(5): 2213-2226.
- [12] ZHOU H, ZHANG C, SONG W, et al. Segmented CRC-aided SC list polar decoding[C]. 2016 83rd Vehicular Technology Conference, Nanjing, China, 2016: 1-5.

- [13] YU Q, SHI Z, YAN Q, et al. Hybrid parity-check and CRC aided SCL decoding for polar codes [C]. 2016 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), Chengdu, China, 2016: 711-716.
- [14] DAI B, GAO C, YAN Z, et al. Parity check aided SC-flip decoding algorithms for polar codes[J]. IEEE Transactions on Vehicular Technology, 2021, 70(10): 10359-10368.
- [15] YUAN F, TIAN B. Double-parity-check CA-SCL encoding and decoding for polar codes[C]. 2018 14th IEEE International Conference on Signal Processing (ICSP), Beijing, China, 2018: 747-751.
- [16] HASHEMI S A, CONDO C, GROSS W J. Simplified Successive-Cancellation List decoding of polar codes[C]. IEEE International Symposium on Information Theory (ISIT), Barcelona, Spain, 2016: 815-819.
- [17] HASHEMI S A, CONDO C, GROSS W J. Fast simplified successive-cancellation list decoding of polar codes[C]. 2017 IEEE Wireless Communications and Networking Conference Workshops (WCNCW), San Francisco, USA, 2017: 1-6.
- [18] HASHEMI S A, CONDO C, GROSS W J. Fast and flexible successive-cancellation list decoders for polar codes[J]. IEEE Transactions on Signal Processing, 2017, 65(21): 5756-5769.
- [19] HANIF M, ARDAKANI M. Fast Successive-Cancellation Decoding of Polar Codes: Identification and Decoding of New Nodes[J]. IEEE Communications Letters, 2017, 21(11): 2360-2363.
- [20] ARDAKANI M H, HANIF M, ARDAKANI M. Fast successive-cancellation-based decoders of polar codes[J]. IEEE Communications Letters, 2019, 67(7): 4562-4574.
- [21] ZHENG H, HASHEMI S A, BALATSOUKAS-STIMMING A, et al. Threshold-based fast successive-cancellation decoding of polar codes[J]. IEEE Transactions on Communications, 2021, 69(6): 3541-3555.
- [22] FEDOR I, VALERII M, EVGENII K. Improved generalized successive cancellation list flip decoder of polar codes with fast decoding of special nodes[J]. Communications and Networks, 2021, 23(6): 417-432.
- [23] KESTEL C, JOHANNSEN L, GRIEBEL O, et al. A 506Gbit/s Polar Successive Cancellation List Decoder with CRC[C]. 2020 IEEE 31st Annual International Symposium on Personal, Indoor and Mobile Radio Communications, London, UK, 2020: 1-7.

- [24] JOHANNSEN L, KESTEL C, GRIEBEL O, et al. Partial Order-Based Decoding of Rate-1 Nodes in Fast Simplified Successive-Cancellation List Decoders for Polar Codes[J]. *Electronics*, 2022, 11(4): 560-588.
- [25] JOHANNSEN L, KESTEL C, VOGT T, et al. Parallel Single Parity Check Nodes for High-Throughput Fast-SCL Polar Code Decoders[C]. 2022 IEEE Symposium on Future Telecommunication Technologies (SOFTT), Johor Baharu, Malaysia, 2022: 28-34.
- [26] ZHAO Y, YIN Z, WU Z, et al. Minimum-Combinations Set-Based Rate-1 Decoder for Fast List Decoding of Polar Codes[J]. *IEEE Communications Letters*, 2021, 25(10): 3185-3189.
- [27] LIU X, WU S, WANG Y. Exploiting error-correction-CRC for polar SCL decoding: a deep learning-based approach[J]. *IEEE Transactions on Cognitive Communications and Networking*. 2020, 6(2): 817-828.
- [28] WANG C, PAN Y, LIN Y. Post-processing for CRC-aided successive cancellation List decoding of polar codes[J]. *IEEE Communications Letters*, 2020, 24(7): 1395-1399.
- [29] FEDOR I, VALERII M, EVGENII K. Improved generalized successive cancellation list flip decoder of polar codes with fast decoding of special nodes[J]. *Communications and Networks*, 2021, 23(6): 417-432.
- [30] SHEN Y, BALATSOUKAS-STIMMING A, YOU X, et al. Dynamic SCL decoder with path-flipping for 5G polar codes[J]. *IEEE Wireless Communications Letters*. 2022, 11(2): 391-395.
- [31] YOU W, YUAN J, YU L. An improved AS-SCLF decoding algorithm of polar codes based on the assigned set[J]. *Optoelectronics Letters*. 2022, 18(11): 694-698.
- [32] BAO J, LIN S, LIU X. An improved successive cancellation list flip decoder for polar codes based on key sets[C]. 15th International Symposium on Medical Information and Communication Technology (ISMICT), Xiamen, China, 2021: 139-144.
- [33] GAO C, LIU R, DAI B, et al. Path splitting selecting strategy-aided successive cancellation List algorithm for polar codes[J]. *IEEE Communications Letters*, 2019, 23(3): 422-425.
- [34] 于永润. 极化码编译码技术研究[D]. 南京: 东南大学, 2019: 19-48.
- [35] LIU H, SHA J, WANG X. An Improved Critical Set for List Decoding of Polar Codes[J]. *IEEE Communications Letters*, 2023, 27(9): 2269-2273.
- [36] WANG X, ZHANG H, LI J, et al. An improved path splitting decision-aided SCL decoding algorithm for polar codes[J]. *IEEE Communications Letters*, 2021, 25(11): 3463-3467.
- [37] SONG W, ZHANG C, ZHANG S, et al. Efficient adaptive successive cancellation list decoders for polar codes[C]. 2016 IEEE International Conference on Digital Signal Processing (DSP), Beijing, China, 2016: 218-222.

- [38] LV Y, YIN H, WANG Y. An Adaptive Ordered Shifted-Pruning List Decoder for Polar Codes[J]. IEEE Access, 2020, 8: 225181-225190.
- [39] ZHANG J, XU H, ZHANG L, et al. Parity-check aided adaptive successive cancellation list decoding for polar Codes[C]. 2021 IEEE 21st International Conference on Communication Technology (ICCT), Tianjin, China, 2021: 138-142.
- [40] YU Y, PAN Z, LIU N, et al. Successive cancellation list bit-flip decoder for polar codes[C]. 2018 10th International Conference on Wireless Communications and Signal Processing (WCSP), Hangzhou, China, 2018: 1-6.
- [41] 王华华, 徐勇军, 秦红, 等. 残差扰动网络辅助的 BP 译码算法[J]. 电讯技术, 2022, 62(8): 1161-1165.
- [42] ZHANG M, LI Z, XING L. An enhanced belief propagation decoder for polar codes[J]. IEEE Communications Letters, 2021, 25(10): 3161-3165.
- [43] ZHANG S, SHAHRAVA B. BP polar-decoding without SNR estimation[C]. 2021 IEEE 12th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON), Vancouver, Canada, 2021: 0194-0197.
- [44] 李秉豪. 极化码的高效 BP 译码算法研究[D]. 西安: 西安电子科技大学, 2021: 27-49.
- [45] ZHOU H, ZHENG J, YANG M, et al. Low-complexity sphere decoding for polar-coded MIMO systems[J]. IEEE Transactions on Vehicular Technology, 2023, 72(5): 6810-6815.
- [46] ZHOU H, DENG X, CAI Y, et al. A Synchro-Set-Aided Breadth-First Sphere Decoder for Polar-Coded MIMO Systems[J]. IEEE Transactions on Signal Processing, 2022, 70: 6200-6215.
- [47] WANG R, CHEN H, CHEN Y, et al. Received Value Flipping Based Sphere Decoding Algorithm for Polar Codes[J]. Intelligent and Converged Networks, 2024, 5(4): 370-379.
- [48] 陈发堂, 赵昊明, 石丹, 等. 低时间复杂度的极化码译码算法[J]. 重庆邮电大学学报 (自然科学版), 2021, 33(04): 571-576.
- [49] FENG H, XIAO H, ZHONG S, et al. Deep-learning-aided fast successive cancellation decoding of polar codes[J]. Journal of Communications and Networks, 2024, 26(6): 593-602.
- [50] LI J, ZHOU L, LI Z, et al. Deep Learning-Assisted Adaptive Dynamic-SCLF Decoding of Polar Codes[J]. IEEE Transactions on Cognitive Communications and Networking, 2024, 10(3): 836-851.
- [51] LEE Y, LEE U, FISSEHA H H, et al. Deep Learning aided BP-Flip Decoding of Polar Codes[C]. 2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS), Incheon, Republic of Korea, 2022: 114-117.

-
- [52] LIANG F S, Lu S, UENG Y L. Deep-Learning-Aided Successive Cancellation List Flip Decoding for Polar Codes[J]. IEEE Transactions on Cognitive Communications and Networking, 2024, 10(2): 374-386.
- [53] WANG X, ZHANG H, TONG J, et al. Perturbation-Enhanced SCL Decoder for Polar Codes[C]. 2023 IEEE Globecom Workshops (GC Wkshps), Kuala Lumpur, Malaysia, 2023: 1674-1679.
- [54] YANG Z, CHEN L, QIN K, et al. Perturbation-Based Decoding Schemes for Long Polar Codes[C]. 2025 IEEE International Symposium on Information Theory (ISIT), Ann Arbor, USA, 2025: 1-6.
- [55] ARLI A C, GAZI O. Noise-Aided Belief Propagation List Decoding of Polar Codes[J]. IEEE Communications Letters, 2019 23(8): 1285-1288.
- [56] ZHANG Z, QIN K, ZHANG L, et al. Progressive bit-flipping decoding of polar codes over layered critical sets[C]. 2017 IEEE Global Communications Conference, Singapore, 2017: 1-6.

作者简介

1. 基本情况

杨德意，男，重庆人，2001 年 1 月出生，重庆邮电大学电子科学与技术专业 2023 级硕士研究生。

2. 教育和工作经历

2019.09~2023.06 重庆邮电大学光电工程学院，本科，专业：电子科学与技术

2023.09~2026.06 重庆邮电大学电子科学与工程学院，硕士研究生，专业：电子科学与技术

3. 攻读学位期间的研究成果

3.1 发表的学术论文和著作

- [1] 袁建国, 杨德意, 向锦能. 基于可靠性函数的极化码 PSS-RF-SCL 译码算法[J]. 半导体光电. 2025, (46): 515-521.
- [2] YUAN Jianguo, YANG Deyi, ZHU Ziliang, et al. Study on the aided PC-ASCL decoding algorithm based on optimized parity check codes[J]. Optoelectronics Letters, 2026, 22(2): 87-91.(EI 检索)

3.2 申请（授权）专利

- [1] 袁建国, 杨德意, 朱子良, 翟少秋. 一种基于优化奇偶校验码辅助极化码 PC-ASCL 译码方法: 202410784353.9[P]. (受理)
- [2] 袁建国, 杨德意, 朱子良, 向锦能. 一种基于可靠性函数的极化码 PSS-RF-SCL 译码方法: 202411658392.0[P]. (受理)

3.3 参与的科研项目及获奖

- [1] 2023 年 校级 研究生一等奖学金
- [2] 2024 年 校级 研究生三等奖学金

致 谢

回首在重庆邮电大学的七年光阴，从 2019 年初秋到如今，仿佛只是一瞬，却又在我的人生中刻下了最深的印记。这七年，是汲取知识的七年，也是收获成长的七年。在此，谨向所有陪伴和帮助过我的人表达衷心的感谢。

感谢母校重庆邮电大学。七年来，从图书馆到实验室，从教学楼到校园的每一个角落，都留下了我奋斗的足迹。感谢学校提供的优质资源和优美环境，让我得以潜心向学，度过人生中最宝贵的青春时光。

饮水思源，感念师恩。特别感谢我的导师袁建国教授。无论是在科研探索中的迷惘之际，还是在论文撰写的关键之时，袁老师总是不厌其烦地指点迷津，给予我热忱的鼓励。在生活中，袁老师亦关怀备至，言传身教为人处世之道；在我稍有松懈之时，又以警醒之语鞭策我保持危机意识、砥砺前行。师恩深重，铭记于心。惟愿恩师身体康健，诸事顺遂。

感谢实验室的兄弟姐妹们。感谢伏博文、杨婷、朱子良、苏杰、苏畅等同学在课题研究中的并肩作战，与你们的讨论让我获益良多。感谢马嘉庆、徐一伟等师兄师姐的耐心解答与无私帮助。感谢胡峻铭、向锦能、胡坤、陈思宏、熊龙宇、邓娅丽等师弟师妹的陪伴与支持，祝你们科研顺利，前程似锦。

感谢我的父母和家人。感谢你们二十多年来的养育之恩，感谢你们为我创造无忧的学习环境，感谢你们在我迷茫时的开导与低谷时的陪伴。你们的支持，是我一路前行的最大动力。

最后，衷心感谢各位评审专家在百忙之中评阅本文，您的宝贵意见将使本研究更加完善。